

Langage Go

Laurent Bellon

2026-2027

Go : Introduction et Fondations

Origines et Motivation

- **Créateurs et Contexte :**

- Conçu en 2007 chez Google par Robert Griesemer, Rob Pike et Ken Thompson (co-inventeur d'UNIX).
- Réponse aux défis des projets Google.

- **Pourquoi Go ? Les Limites du C++ et Python :**

- **C++ :** Complexité, lenteur de compilation, maintenance difficile pour de grandes bases de code.
- **Python :** Développement rapide mais manque de performance et de rigueur pour la production à grande échelle.

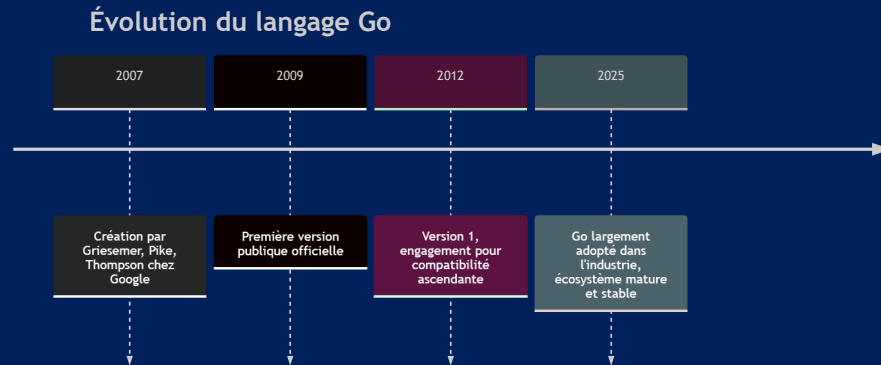
Go : Une Réponse aux Défis des Logiciels Modernes

Go a été conçu pour offrir :

- **Simplicité et clarté** dans la syntaxe.
- **Vitesse d'exécution** comparable aux langages compilés (C++).
- Un modèle de **concurrence performant** via les "goroutines".
- Une gestion moderne des dépendances et une **compilation rapide**.
- Une **compatibilité ascendante** garantie depuis sa version 1 (2012).

Go combine les avantages des langages compilés et la productivité des langages modernes.

Évolution du Langage Go : Une Chronologie



Go en Action : Hello World et Clarté

Exemple : Hello World en Go

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, Go!")
}
```

Explication :

- `package main` : Définit un programme exécutable.
- `import "fmt"` : Importe le package standard pour l'affichage.
- `func main` : La fonction principale, point d'entrée du programme.
- `fmt.Println` : Affiche une chaîne de caractères à l'écran.

Go en Action : La Concurrency via les Goroutines

Exemple : Concurrency simplifiée avec les goroutines

```
package main

import (
    "fmt"
    "time"
)

func say(s string) {
    for i := 0; i < 3; i++ {
        fmt.Println(s)
        time.Sleep(100 * time.Millisecond)
    }
}

func main() {
    go say("Hello")
    say("World")
}
```

Explication :

- `go say("Hello")` : Lance la fonction `say` dans une goroutine (thread léger).
- Le programme exécute "Hello" et "World" de manière intercalée, illustrant la gestion native de la concurrence.

Go : Résumé et Ressources

Ce qu'il faut retenir :

- Go a été conçu pour répondre aux besoins contemporains des développeurs de logiciels complexes à grande échelle.
- Il met l'accent sur la simplicité et la robustesse.
- Intègre des outils pour la gestion de la concurrence, la rapidité de compilation et une évolution maîtrisée.

Sources :

- Encyclopaedia Britannica, "Go | Programming Language, Features, & Facts": <https://www.britannica.com/technology/Go-programming-language>
- Tencent Cloud Techpedia, "What is the history and background of the Go language?": <https://www.tencentcloud.com/techpedia/101475>
- ByteSizeGo, "The Evolution of the Go Programming Language": <https://www.bytesizego.com/blog/go-language-history>
- ACM Communications, "The Go Programming Language and Environment": <https://cacm.acm.org/research/the-go-programming-language-and-environment>
- GeeksforGeeks, "Go Programming Language (Introduction)": <https://www.geeksforgeeks.org/go-language/go-programming-language-introduction>

Introduction à Go : Simplicité et Performance

- **Présentation du langage Go**

- Un langage moderne, développé par Google, apparu pour répondre aux défis du développement logiciel à grande échelle.
- Axé sur la performance, la simplicité et la fiabilité.

- **Avantages clés de Go**

- **Simplicité et lisibilité du code :**

- Syntaxe claire et minimaliste.
- Évite les complexités inutiles (pas d'héritage multiple, d'exceptions, de macros).
- Le code Go est facile à lire, comprendre et maintenir.

- **Performance proche du langage natif :**

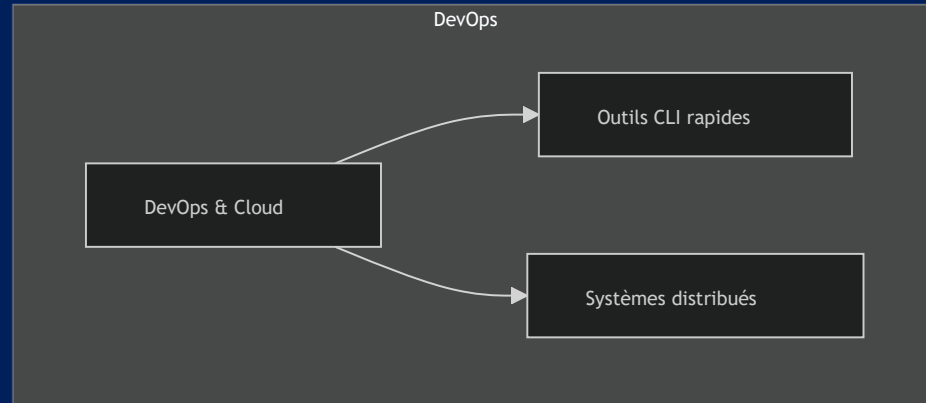
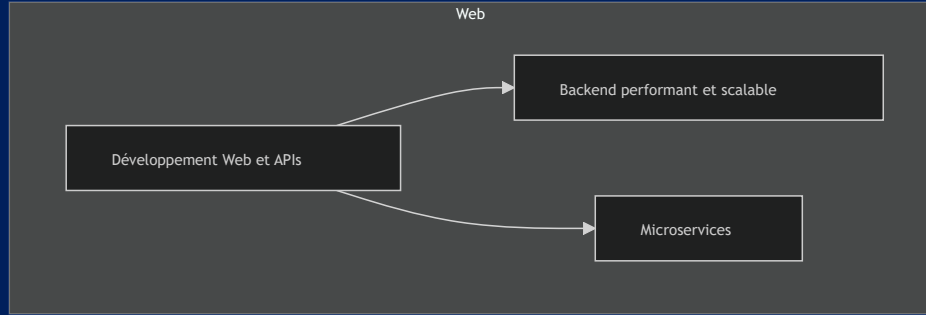
- Langage compilé générant du code machine optimisé.
- Atteint des performances comparables à celles du C/C++.
- Compilation rapide, même pour de gros projets.

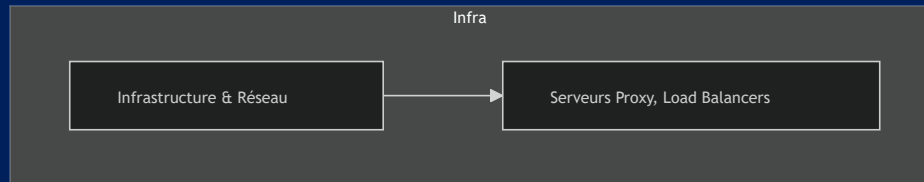
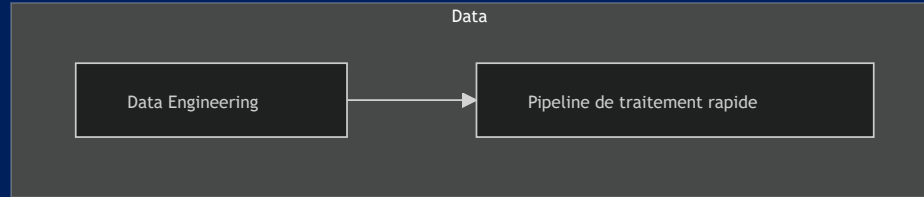
Go : Concurrency, Écosystème et Déploiement

- **Gestion efficace de la concurrence :**
 - Modèle basé sur les *goroutines* — des threads légers gérés par le runtime Go.
 - Communication entre goroutines via des *canaux* ("channels"), simplifiant la programmation parallèle.
 - Réduit les risques classiques des threads (deadlocks, conditions de course).
- **Écosystème riche et standardisé :**
 - Bibliothèque standard puissante (réseau, fichiers, cryptographie, etc.).
 - Système de gestion des dépendances (modules Go) simple et intégré.
- **Compilation multi-plateforme :**
 - Le compilateur Go peut cross-compiler facilement pour différents OS et architectures CPU (Linux, Windows, macOS, ARM, etc.).
 - Pas de configuration complexe.
- **Déploiement simplifié grâce aux binaires statiques :**
 - Produit des exécutables statiques incluant toutes les dépendances nécessaires.
 - Facilite le déploiement sans gérer de runtime ou bibliothèques externes.

Cas d'Utilisation Stratégiques de Go

- **Go est privilégié pour :**
 - **Développement Web et APIs :** Backends performants et scalables, microservices.
 - **DevOps & Cloud :** Outils CLI rapides, systèmes distribués.
 - **Data Engineering :** Pipelines de traitement de données rapides.
 - **Infrastructure & Réseau :** Serveurs Proxy, Load Balancers.
- **Vue d'ensemble des domaines d'application :**





Go en Action : Web, API et Infrastructures

- **Serveurs Web et API performants :**

- Go est largement utilisé pour construire des serveurs web gérant des milliers de connexions simultanées avec une faible latence.
- Grâce à ses goroutines et sa gestion efficace des requêtes réseau.

- **Exemple de serveur HTTP basique :**

```
package main

import (
    "fmt"
    "net/http"
)

func handler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Hello, %s!", r.URL.Path[1:])
}

func main() {
    http.HandleFunc("/", handler)
    http.ListenAndServe(":8080", nil)
}
```

- **Microservices :**

- Simplicité, rapidité de compilation et gestion naturelle de la concurrence font de Go un excellent choix pour développer des microservices légers et faciles à maintenir.

- **Cloud et infrastructures :**

- De nombreux projets Cloud natifs (Kubernetes, Docker, Terraform) sont écrits en Go.
- Preuve de son efficacité pour les systèmes distribués, l'orchestration et les infrastructures.

Maîtriser la Concurrency avec Goroutines & Channels

Illustration de la gestion de la concurrence :

```
func worker(id int, c chan string) {  
    for i := 0; i < 3; i++ {  
        msg := fmt.Sprintf("Worker %d - tâche %d", id, i)  
        c <- msg  
        time.Sleep(time.Millisecond * 100)  
    }  
    close(c)  
}  
  
func main() {  
    c := make(chan string)  
  
    go worker(1, c)  
  
    for msg := range c {  
        fmt.Println(msg)  
    }  
}
```

Ce code démontre comment les goroutines et canaux simplifient la communication et la synchronisation entre tâches concurrentes.

Go : Un Choix Stratégique pour le Développement

- **Ce qu'il faut retenir :**

- Go offre une combinaison unique de **simplicité, performance native et gestion efficace de la concurrence**.
- Son **écosystème riche**, sa **compilation multi-plateforme** et son **déploiement simplifié** en font un atout majeur.
- Idéal pour le **développement de serveurs, les microservices, les outils en ligne de commande (CLI) et les infrastructures cloud**.
- Sa popularité croissante s'explique par son efficacité dans le développement, particulièrement dans l'écosystème cloud et la programmation serveur.

- **Sources utilisées :**

- Go Language Official site, "Why Go?": https://go.dev/doc/faq#why_go
- The New Stack, "Why Google Created Go": <https://thenewstack.io/why-google-created-go-programming-language/>
- GitHub, Kubernetes repo (exemple d'application Go dans le cloud) : <https://github.com/kubernetes/kubernetes>
- DigitalOcean, "A Beginner's Guide to Go": https://www.digitalocean.com/community/tutorial_series/a-beginner-s-guide-to-go
- Medium, "Real-world Use Cases for Go Language": <https://medium.com/@amit.thewheelers/real-world-use-cases-for-go-f2e1f247ccc3>
- Go By Example, "Goroutines and Channels": <https://gobyexample.com/goroutines>

Installer Go

1. Va sur la page officielle : <https://go.dev/dl/>
2. Télécharge le **Windows installer (.msi)** correspondant à ta machine (probablement **amd64**).
3. Lance l'installateur → il va ajouter `go` et `gofmt` à ton **PATH**.
4. Vérifie l'installation dans **PowerShell** ou **cmd** :

```
go version
```

Tu dois voir un message du type `go version go1.23.2 windows/amd64` .

Choisir un IDE / Éditeur

Plusieurs options selon ton style :

- **Visual Studio Code (recommandé)**
 - Installe VS Code : <https://code.visualstudio.com>
 - Ajoute l'extension **Go** officielle (par l'équipe Go).
 - Elle fournit : autocomplétion, refactoring, lint, debug, test.
- **GoLand (JetBrains, payant mais très complet)**
 - Excellente intégration, debugging puissant.
 - Version gratuite dispo pour étudiants.
- **LiteIDE (léger, open-source)**
 - Moins populaire aujourd'hui, mais très simple si tu veux un IDE spécialisé Go.

Étape 3 – Configurer le Debugger

Le débogueur standard pour Go est **Delve**.

- Si tu utilises VS Code, l'extension **Go** installe automatiquement Delve (`dlv`) pour toi.
- Tu pourras alors poser des breakpoints, inspecter les variables et exécuter pas à pas.

Étape 4 – Organisation du Workspace

1. Choisis un dossier de travail, par exemple `C:\Users\TonNom\go` .

2. Vérifie ton GOPATH :

```
go env GOPATH
```

Par défaut, c'est `C:\Users\TonNom\go` .

3. Initialise un projet :

```
mkdir hello-go  
cd hello-go  
go mod init exemple.com/hello
```

Cela crée un fichier `go.mod` .

Étape 5 – Tests unitaires

Go intègre directement un framework de test :

```
package main

import "testing"

func TestAddition(t *testing.T) {
    if 2+2  $\neq$  4 {
        t.Errorf("Attendu 4, obtenu %d", 2+2)
    }
}
```

Exécute :

```
go test ./...
```

Étape 6 – Outils utiles

- `go fmt` → formate ton code.
- `go vet` → détecte des erreurs potentielles.
- `golangci-lint` (à installer en plus) → linting avancé.
- `air` (si tu fais du web Go) → rechargement automatique du serveur.

Résumé rapide :

- **Compilateur** → fourni avec Go (`go build` , `go run`).
- **IDE** → VS Code + extension Go (ou GoLand si tu veux du full IDE).
- **Debugger** → Delve (intégré avec VS Code).
- **Tests** → inclus (`go test`).

Entrée et Sortie Simples en Go

- 1- Introduction au Go & bases du langage
- 2- Bases du langage Go
- 3- Entrée et sortie simples (print, scan)

Voici les commandes fondamentales des entrées/sorties en Go, permettant d'échanger simplement avec l'utilisateur via la console.

Affichage de Données : Le Package `fmt`

En Go, le package standard `fmt` (format) fournit les fonctions les plus courantes pour l'affichage.

1. Affichage simple avec `fmt.Print` et `fmt.Println`

- `fmt.Print()` : Affiche les arguments sans ajouter de saut de ligne.
- `fmt.Println()` : Affiche les arguments et ajoute un saut de ligne automatiquement.

```
package main

import "fmt"

func main() {
    fmt.Print("Bonjour ")
    fmt.Print("le monde!")
    fmt.Println(" Ceci sera sur la même ligne.")
    fmt.Println("Et ici, on passe à la ligne suivante.")
}
```

Sortie :

```
Bonjour le monde! Ceci sera sur la même ligne.
Et ici, on passe à la ligne suivante.
```

2. Affichage formaté avec `fmt.Printf`

- Permet d'utiliser des verbes de formatage (`%d` pour entier, `%s` pour chaîne, `%f` pour flottant, etc.).

```
fmt.Printf("Valeur entière : %d, texte : %s, flottant : %.2f\n", 42, "exemple", 3.14159)
```

Sortie :

```
Valeur entière : 42, texte : exemple, flottant : 3.14
```

Lecture de Données : `fmt.Scan` et Variantes

Le package `fmt` inclut des fonctions pour lire l'entrée standard (clavier).

1. `fmt.Scan`

- Lit les entrées séparées par des espaces.
- Les stocke dans des variables passées en *référence* (`&`).

```
package main

import "fmt"

func main() {
    var name string
    var age int

    fmt.Print("Entrez votre nom : ")
    fmt.Scan(&name) // & pour passer l'adresse

    fmt.Print("Entrez votre âge : ")
    fmt.Scan(&age)

    fmt.Printf("Nom: %s, Âge: %d\n", name, age)
}
```

2. Variantes utiles

- `fmt.Scanln` : Lit jusqu'à la fin de la ligne. Pratique pour lire une seule ligne complète.
- `fmt.Sscan` : Pour lire depuis une *chaîne* au lieu de l'entrée standard.

Gestion des Erreurs et Exemple Combiné

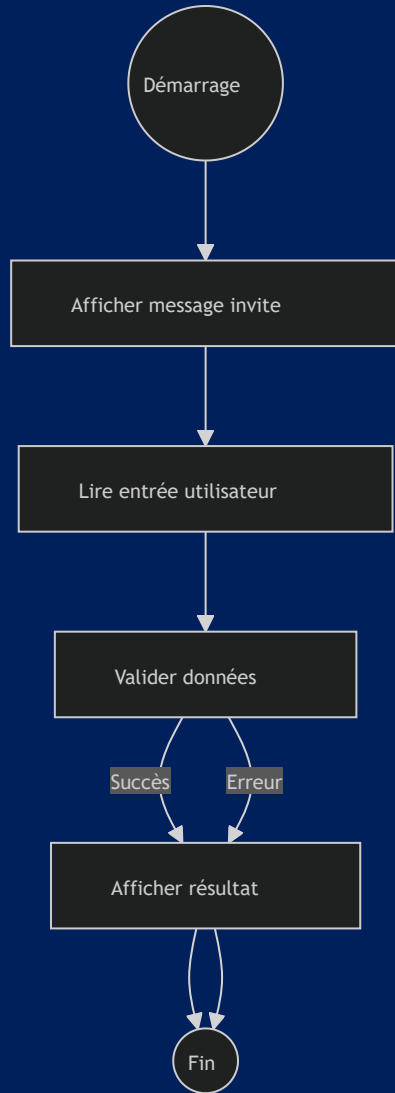
1. Gestion des erreurs de lecture

- `fmt.Scan` retourne le nombre d'éléments lus et une `error` qu'il faut vérifier.

```
n, err := fmt.Scan(&name)
if err != nil {
    fmt.Println("Erreur lors de la lecture :", err)
    return
}
fmt.Printf("Valeurs lues : %d\n", n)
```


Processus Général d'Entrée/Sortie

Visualisation du flux d'interaction avec l'utilisateur pour les opérations d'entrée/sortie.



Ce Qu'il Faut Retenir & Ressources

Ce qu'il faut retenir :

- Les fonctions `fmt.Print` , `fmt.Println` , `fmt.Printf` sont essentielles pour l'affichage console.
- `fmt.Scan` et ses variantes permettent une lecture simple des entrées utilisateur.
- La gestion des erreurs (`if err != nil`) est cruciale pour la robustesse des programmes.
- Ces commandes fondamentales facilitent l'échange simple avec l'utilisateur via la console.

Sources utiles :

- Documentation Go, package fmt: <https://pkg.go.dev/fmt>
- Tour of Go, "Basic Data Types": <https://go.dev/tour/basics/10>
- Go by Example, "Scanning user input": <https://gobyexample.com/reading-input>
- Stack Overflow, "Best way to read user input in Go": <https://stackoverflow.com/questions/28070899/how-to-read-input-from-console-in-golang>

Go : Déclaration de variables et types fondamentaux

Déclarer vos variables en Go : Syntaxes et Portée

1. Syntaxes principales

Syntaxe classique avec `var` et typage explicite

```
var a int = 10  
var b string = "Hello"
```

Déclaration avec inférence de type

```
var a = 10           // a est de type int  
var b = "World"      // b est de type string
```

Go déduit le type à partir de la valeur assignée.

Déclaration courte (usage fréquent dans les fonctions)

```
a := 10  
b := "Hello !"
```

Cette syntaxe fonctionne uniquement à l'intérieur des fonctions.

2. Portée et utilisation

- Les variables déclarées avec `var` peuvent avoir une portée **globale** ou **locale**.
- La déclaration courte `:=` est **strictement locale**.

Types Fondamentaux (1/2) : Entiers et Flottants

Go est un langage strictement typé.

a) Type entier (`int`)

- Plusieurs tailles : `int8`, `int16`, `int32`, `int64`. La taille de `int` dépend de la plateforme.
- Signés par défaut. Pour les entiers **non signés**, utilisez `uint` et variantes (`uint8` équivaut à `byte`).

Exemple :

```
var age int = 30
var score uint16 = 65535 // Entier non signé
```

b) Type flottant (`float32`, `float64`)

- `float32` ou `float64` pour les nombres à virgule.
- `float64` (double précision) est recommandée pour la plupart des calculs.

Exemple :

```
var price float64 = 19.99
var rate float32 = 0.07
```

Types Fondamentaux (2/2) : Chaînes et Booléens

c) Type chaîne de caractères (`string`)

- **Immuables** (ne peuvent pas être modifiées après création).
- Encodage **UTF-8** pour les caractères Unicode.
- Guillemets doubles (`"`) pour chaînes simples, backquotes (```) pour chaînes brutes multi-lignes.

Exemple :

```
var message string = "Bonjour, 世界"  
rawString := `Chaîne  
sur  
plusieurs lignes`
```

d) Type booléen (`bool`)

- Deux valeurs possibles : `true` ou `false` .
- Utilisé pour les conditions et flags.

Exemple :

```
var isActive bool = true  
var hasErrors bool // false par défaut
```

Mise en Pratique & Cycle de Vie des Variables

Exemple complet

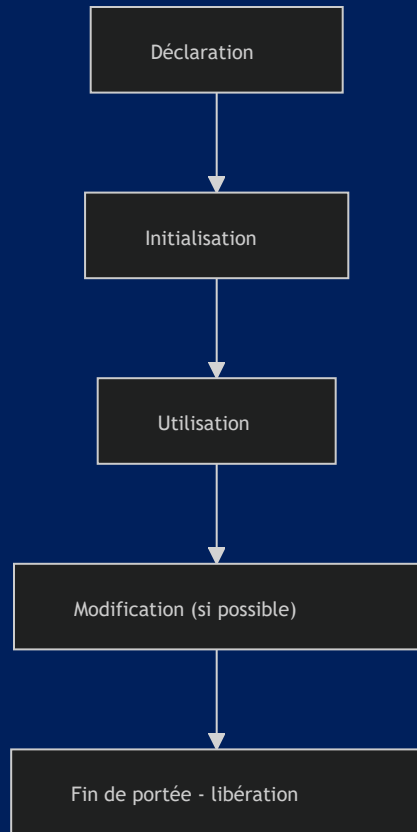
```
package main

import "fmt"

func main() {
    var i int = 42      // Syntaxe classique
    j := 3.14           // Déclaration courte, inférence float64
    var s string = "Go" // Syntaxe classique
    b := false          // Déclaration courte, inférence bool

    fmt.Println("Entier:", i)
    fmt.Println("Flottant:", j)
    fmt.Println("Chaîne :", s)
    fmt.Println("Booléen:", b)
}
```

Cycle de vie d'une variable en Go



Cycle de vie d'une variable en Go

- La variable est déclarée et éventuellement initialisée.
- Elle est utilisée et peut être modifiée.
- Lorsque la variable sort de sa portée, Go gère automatiquement la libération mémoire grâce au ramasse-miettes (GC).

Sources

- Documentation officielle Go, "Variables": https://go.dev/doc/effective_go#variables
- Tour of Go, "Variables and Types": <https://tour.golang.org/basics/2>
- Golang Bot, "Go Data Types": <https://golangbot.com/types/>
- Go by Example, "Variables": <https://gobyexample.com/variables>
- Go Blog, "Introduction to Go": <https://blog.golang.org/go-at-google>

Structures de Contrôle en Go

`if`, `for`, `switch`

- Les bases pour maîtriser le flux d'exécution
- Clarté et efficacité dans vos programmes Go

La structure conditionnelle `if`

L'instruction `if` évalue une condition booléenne et exécute un bloc de code si elle est vraie.

Syntaxe simple

```
if condition {  
    // instructions exécutées si condition est vraie  
}
```

Exemple

```
package main  
import "fmt"  
  
func main() {  
    age := 20  
    if age ≥ 18 {  
        fmt.Println("Adulte") // Affiche "Adulte"  
    }  
}
```


La boucle `for`

`for` est la *seule* structure de boucle en Go, très flexible.

a) `for` **classique** Comporte trois parties : initialisation, condition, post-instruction.

```
for i := 0; i < 5; i++ {  
    fmt.Println(i) // Affiche 0, 1, 2, 3, 4  
}
```

b) `for` **type** `while` Sans initialisation ni post-instruction.

```
i := 0  
for i < 5 {  
    fmt.Println(i)  
    i++  
}
```

c) **Boucle infinie**

```
for {  
    // boucle infinie (à utiliser avec 'break')  
}
```

La structure `switch`

Teste une variable contre plusieurs cas possibles pour une syntaxe plus claire que de multiples `if-else` .

Syntaxe basique

```
switch variable {  
  case valeur1:  
    // bloc 1  
  case valeur2:  
    // bloc 2  
  default:  
    // bloc par défaut  
}
```

Exemple

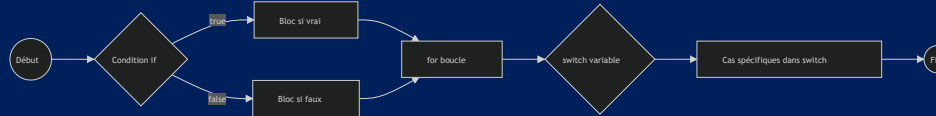
```
package main
import "fmt"

func main() {
    day := "mardi"
    switch day {
    case "lundi":
        fmt.Println("Début de la semaine")
    case "mardi", "mercredi": // Cas multiples
        fmt.Println("Milieu de semaine") // Affiche "Milieu de semaine"
    case "samedi", "dimanche":
        fmt.Println("Weekend!")
    default:
        fmt.Println("Jour inconnu")
    }
}
```

Particularités

- Pas besoin de `break` explicite (pas de "fall-through" par défaut).
- On peut utiliser `fallthrough` pour forcer l'exécution du cas suivant.

Synthèse Visuelle des Structures



Structure	Remarques	Syntaxe importante
<code>if</code>	Permet une déclaration avant condition	<code>if v := expr(); v > 0 {}</code>
<code>for</code>	La seule boucle, variations multiples	<code>for i:=0;i<10;i++ {}</code> , <code>for condition {}</code>
<code>switch</code>	Pas besoin de <code>break</code> , cas multiples	<code>switch x { case 1,2: ... default: ... }</code>

Ce qu'il faut retenir & Sources

Ce cours présente l'essentiel des structures de contrôle dans Go. Leur compréhension permet de maîtriser le flux d'exécution dans vos programmes avec clarté et efficacité.

Sources

- Documentation officielle Go, "Flow control": https://go.dev/doc/effective_go#control-structures
- Tour of Go, "Flow control statements": <https://go.dev/tour/controlflow/1>
- Go by Example, "If/Else": <https://gobyexample.com/if-else>
- Go by Example, "For loop": <https://gobyexample.com/for>
- Go by Example, "Switch": <https://gobyexample.com/switch>

Collections en Go : Tableaux & Slices

Tableaux en Go : Fondamentaux

Un **tableau** en Go est une séquence d'éléments de **taille fixe** et de même type.

Déclaration & Caractéristiques

```
var arr [5]int           // tableau de 5 entiers initialisés à 0
arr2 := [3]string{"a", "b", "c"} // déclaration et initialisation
```

- **Taille fixe** et fait partie du type (`[5]int` $\neq$ `[10]int`).
- Taille connue à la compilation.

Accès, Modification & Parcours

```
arr[0] = 42
fmt.Println(arr[0]) // affiche 42

// Parcours
for i, v := range arr {
    fmt.Println(i, v)
}
```

Slices : L'Abstraction Dynamique

Les **slices** sont des structures dynamiques, offrant plus de flexibilité que les tableaux.

Qu'est-ce qu'une slice ?

Une slice décrit une portion continue d'un tableau sous-jacent. Elle contient :

- un pointeur vers le tableau,
- une longueur (length),
- une capacité (capacity).

Déclaration et Initialisation

```
var s []int           // slice vide, nil
s2 := make([]int, 5)  // slice de 5 entiers initialisés à 0
s3 := []string{"x", "y", "z"} // slice littérale
```

Conversion Tableau → Slice

```
arr := [4]int{1,2,3,4}
sl := arr[1:3] // slice contenant {2,3}
```

Manipulation des Slices

Accès et Modification

Comme pour les tableaux :

```
s[0] = 10  
fmt.Println(s[0])
```

Extension avec `append`

Les slices sont extensibles via la fonction intégrée `append` :

```
var s []int  
s = append(s, 1)  
s = append(s, 2, 3)
```

‘append’ peut réallouer et copier le tableau sous-jacent si la capacité est dépassée.

Copie de slices

Pour copier le contenu d'une slice dans une autre :

```
src := []int{1,2,3}  
dst := make([]int, len(src))  
copy(dst, src)
```

Illustration Pratique : Code Complet

```
package main

import "fmt"

func main() {
    // Déclaration d'un tableau
    var tab [3]int = [3]int{10, 20, 30}
    fmt.Println("Tableau:", tab)

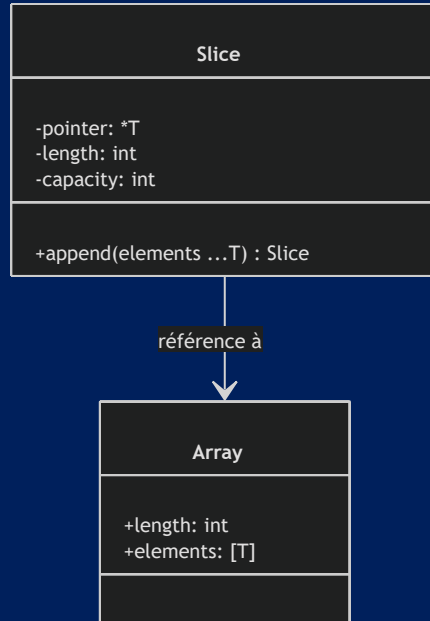
    // Déclaration d'une slice
    slice := []int{1, 2, 3}
    fmt.Println("Slice:", slice)

    // Slice d'un tableau
    s := tab[1:3]
    fmt.Println("Slice sur tableau:", s)

    // Modification slice modifie le tableau sous-jacent
    s[0] = 100
    fmt.Println("Tableau après modification slice:", tab)

    // Append sur slice (peut réallouer)
    slice = append(slice, 4, 5)
    fmt.Println("Slice après append:", slice)
}
```

Structure Relationnelle



Points Clés & Sources

Ce qu'il faut retenir

Concept	Description
Tableau	Taille fixe, allocation statique
Slice	Taille dynamique, référence à un tableau sous-jacent
<code>append</code>	Ajouter des éléments à une slice, possible réallocation
<code>copy</code>	Copier les éléments entre slices

Sources

- Documentation officielle Go, Arrays and Slices: https://go.dev/doc/effective_go#arrays_and_slices
- Tour of Go, "Arrays and Slices": <https://go.dev/tour/moretypes/7>
- Go by Example, "Arrays": <https://gobyexample.com/arrays>
- Go by Example, "Slices": <https://gobyexample.com/slices>
- Stack Overflow, "What is the difference between arrays and slices in Go?": <https://stackoverflow.com/questions/34671245/what-is-the-difference-between-arrays-and-slices-in-go>

Ce cours a présenté l'essentiel des tableaux et slices en Go, fondation des structures de données pour la manipulation efficace des collections.

Les Maps en Go : Stockage Associatif Clé-Valeur

Qu'est-ce qu'une Map ?

Une **map** est une structure de données associative qui stocke des paires clé-valeur. Elle permet un accès rapide aux valeurs à partir de leurs clés.

Syntaxe Générale

```
map[TypeClé]TypeValeur
```

Exemple : `map[int]string` (une map de chaînes indexées par des entiers).

Déclaration et Initialisation des Maps

1. Déclaration d'une map `nil`

Cette déclaration crée une map nulle qui ne peut pas encore être utilisée.

```
var m map[string]int // nil, non initialisée et inutilisable
```

2. Initialisation avec `make`

Pour utiliser une map, il faut l'initialiser :

```
m = make(map[string]int)
```

3. Déclaration et initialisation combinées

```
m := make(map[string]int) // Map vide et utilisable
```

Ou avec des valeurs initiales :

```
m := map[string]int{  
    "a": 1,  
    "b": 2,  
}
```

Accès, Ajout, Modification, Suppression

1. Accès aux éléments

- Lire une valeur :

```
val := m["a"]
```

- Tester l'existence d'une clé :

```
val, ok := m["clé"]  
if ok {  
    fmt.Println("Valeur :", val)  
} else {  
    fmt.Println("Clé non trouvée")  
}
```

2. Ajout ou Modification

```
m["nouvelleclé"] = 10 // Ajoute une nouvelle clé  
m["cléExistante"] = 20 // Modifie la valeur d'une clé existante
```

3. Suppression

```
delete(m, "cléASupprimer")
```

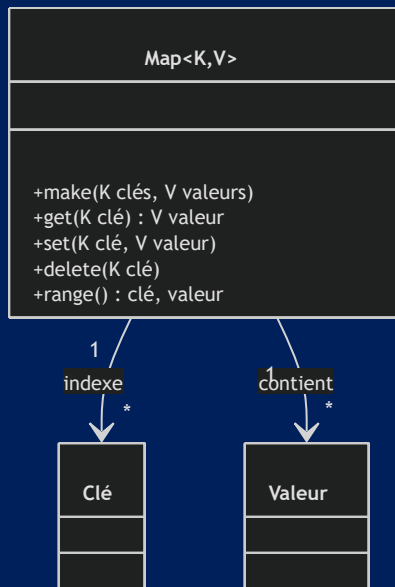
Parcours et Fonctionnement d'une Map

1. Parcours d'une map

```
for clé, val := range m {  
    fmt.Println(clé, ":", val)  
}
```

Note : L'ordre dans lequel les paires sont retournées est non déterministe.

2. Diagramme conceptuel d'une map



Points Clés et Ressources

Ce qu'il faut retenir

Opération	Syntaxe	Notes
Déclaration	<code>var m map[K]V</code>	Initialise à <code>nil</code> , inutilisable avant <code>make</code>
Initialisation	<code>m = make(map[K]V)</code>	Crée une map vide utilisable
Ajout/modif	<code>m[clé] = valeur</code>	Écrase si la clé existe déjà
Accès	<code>val, ok := m[clé]</code>	<code>ok</code> indique si clé existe
Suppression	<code>delete(m, clé)</code>	Supprime la paire clé-valeur
Parcours	<code>for k, v := range m</code>	Itère dans un ordre non garanti

Ce cours explicite la manipulation des maps, structure clé pour l'association rapide clé-valeur dans Go, indispensable pour organiser et accéder efficacement aux données.

Sources

- Go Official Blog, "Maps in Go": <https://go.dev/blog/maps>
- Documentation Go, "Maps": <https://pkg.go.dev/builtin#map>
- Tour of Go, "Maps": <https://go.dev/tour/moretypes/16>
- Go by Example, "Maps": <https://gobyexample.com/maps>

Fonctions en Go : Définition et Appel

Une fonction en Go est une séquence d'instructions qui peut prendre des paramètres, retourner des valeurs et être appelée depuis d'autres parties du programme.

Syntaxe générale

```
func nomFonction(param1 type1, param2 type2) retourType {  
    // corps de la fonction  
    return valeur  
}
```

- `func` introduit la fonction.
- Les paramètres sont listés avec leur nom puis leur type.
- Le type de retour est indiqué après la parenthèse de paramètres.
- Pour une fonction sans valeur de retour, ne rien mettre après la parenthèse.

Construire une première fonction

Exemple simple d'addition

```
package main

import "fmt"

func addition(a int, b int) int { // Définition : prend deux entiers, retourne un entier
    return a + b                  // Corps : réalise la somme
}

func main() {
    res := addition(10, 5)        // Appel : utilise la fonction "addition"
    fmt.Println("Résultat :", res) // Affiche "Résultat : 15"
}
```

Cette fonction illustre comment définir un bloc de code réutilisable pour effectuer une tâche spécifique.

Flexibilité des fonctions : Retours multiples et concision

Plusieurs retours

Go supporte le retour de plusieurs valeurs, fréquemment utilisé pour combiner un résultat et une erreur.

```
func diviser(a, b float64) (float64, error) {  
    if b == 0 {  
        return 0, fmt.Errorf("division par zéro") // Retourne 0 et une erreur  
    }  
    return a / b, nil // Retourne le résultat et nil (pas d'erreur)  
}
```

Paramètres de types similaires

Les paramètres adjacents de même type peuvent être regroupés pour plus de concision.

```
func ajouter(x, y int) int { // Au lieu de func ajouter(x int, y int) int  
    return x + y  
}
```

Interactions : Appel et Fonctions sans résultat

Appel de fonction

Il suffit d'utiliser le nom de la fonction avec les arguments requis.

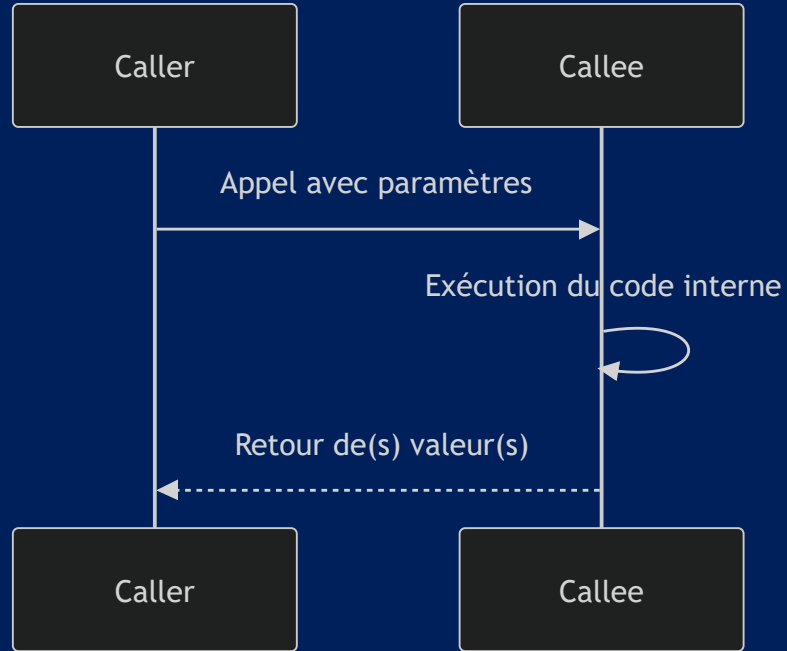
```
resultat := addition(3, 4) // Appel de la fonction 'addition'
```

Fonctions sans retour

Certaines fonctions exécutent une action sans avoir besoin de renvoyer une valeur.

```
func afficherMessage(msg string) { // Pas de type de retour spécifié
    fmt.Println(msg)
}
```

Cycle de vie d'un appel de fonction



Fonctions Anonymes : Flexibilité avancée

Go permet de déclarer des fonctions sans nom (fonctions littérales) et de les affecter à des variables.

Exemple d'affectation

```
f := func(x int) int { // Déclaration d'une fonction anonyme et affectation à 'f'
    return x * x
}
fmt.Println(f(5)) // 25 - Appel via la variable 'f'
```


L'essentiel à retenir & Pour aller plus loin

Résumé des éléments clés

Élément	Description
<code>func</code>	Mot-clé pour définir une fonction
Paramètres	Variables d'entrée de la fonction
Valeur ou valeurs de retour	Ce que la fonction renvoie
Plusieurs retours	Possible via <code>(type1, type2, ...)</code>
Fonctions anonymes	Fonctions sans nom, affectables à des variables
Appel	Nom de la fonction + arguments

Sources

- Tour of Go, "Functions": <https://go.dev/tour/basics/4>
- Documentation officielle Go, "Functions": https://golang.org/ref/spec#Function_declarations
- Go by Example, "Functions": <https://gobyexample.com/functions>
- Go Blog, "Effective Go – Functions": https://golang.org/doc/effective_go#functions

Paramètres de Fonctions en Go : Fondamentaux

- **Syntaxe Basique :**

- Les paramètres sont listés entre parenthèses (nom type, nom type) .
- Le type peut être factorisé pour des paramètres consécutifs de même type.
- Ex: `func addition(a, b int) int`

- **Types de Passage :**

- **Par Valeur :** Une copie de la valeur est transmise à la fonction. La valeur originale n'est pas modifiée.
- **Par Référence (Pointeur) :** L'adresse mémoire est passée (*type). La fonction peut modifier la valeur originale via ce pointeur.

```
func incrementer(x *int) {  
    *x = *x + 1 // Modifie la valeur à l'adresse pointée  
}  
// Utilisation : a := 5; incrementer(&a); // a devient 6
```

Paramètres Variadiques : Flexibilité en Go

- **Concept :**
 - Permet de définir des fonctions acceptant un nombre variable de paramètres du même type.
 - Utilise la syntaxe `... type` .
- **Fonctionnement :**
 - À l'intérieur de la fonction, les paramètres variadiques sont traités comme une "slice" (tranche) du type spécifié.
- **Exemple : Somme d'un nombre variable d'entiers**

```
func somme(nums ...int) int {  
    total := 0  
    for _, n := range nums {  
        total += n  
    }  
    return total  
}  
  
// Utilisation :  
// fmt.Println(somme(1, 2, 3))           // Affiche 6  
// fmt.Println(somme(4, 5, 6, 7, 8))    // Affiche 30
```

Gérer les Retours de Fonctions en Go

- **Retour Simple :**

- Une fonction peut retourner une seule valeur. Le type de retour est indiqué après les parenthèses des paramètres.
- Ex: `func carre(x int) int { return x * x }`

- **Retour Multiple :**

- Go autorise le retour de plusieurs valeurs, très utile pour retourner une valeur et une potentielle erreur.

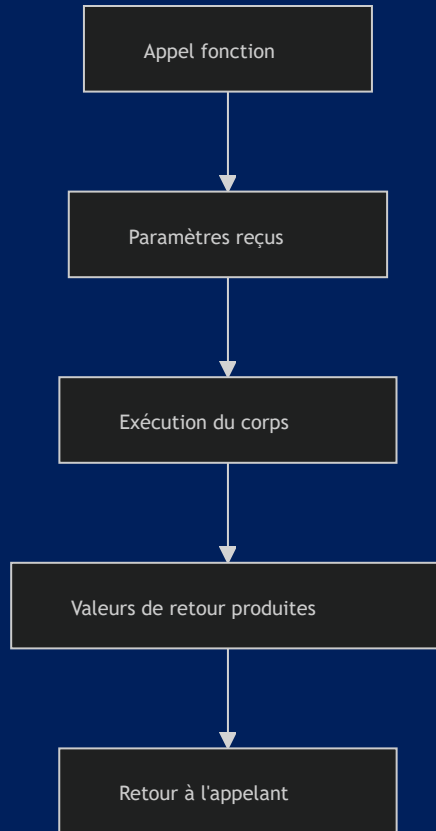
```
func diviser(a, b float64) (float64, error) {  
    if b == 0 {  
        return 0, fmt.Errorf("division par zéro")  
    }  
    return a / b, nil  
}  
  
// Utilisation :  
// result, err := diviser(10, 2) // result=5, err=nil  
// _, err := diviser(10, 0)      // err="division par zéro"
```

- **Nommage des Valeurs de Retour :**

- On peut nommer les variables de retour dans la signature de la fonction.
- Cela permet d'utiliser un simple `return` sans spécifier les valeurs explicitement, celles-ci étant déjà initialisées ou assignées.

```
func longueurEtPremier(s string) (longueur int, premier byte) {  
    longueur = len(s)  
    if longueur > 0 {  
        premier = s[0]  
    }  
    return // Retourne implicitement longueur et premier  
}
```

Comprendre le Flux et l'Intégration



- **Exemple Complet (Synthèse des Concepts) :**

- Les mécanismes de paramètres (par valeur, par pointeur, variadiques) et de retours (simples, multiples, nommés) sont souvent combinés pour créer des fonctions robustes et expressives.
- Le code ci-dessous illustre l'intégration de :
 - `incrémenter(&n)` : utilisation d'un pointeur.
 - `somme(1, 2, 3, 4)` : paramètres variadiques.
 - `diviser(10, 0)` : retours multiples avec gestion d'erreurs.

L'Essentiel à Retenir sur Paramètres et Retours

Aspect	Description
Paramètres	Passage par valeur, par pointeur (adresse), ou variadiques
Retour	Une ou plusieurs valeurs possibles
Nommage des retours	Permet retourner en <code>return</code> simple
Paramètres variadiques	Syntaxe <code>... type</code> , accessibles comme slice

Conclusion et Références Utiles

Ce cours éclaire la variété des paramètres et retours dans Go, outils fondamentaux pour écrire des fonctions expressives, modulaires et sûres.

Sources :

- Tour of Go, "Functions": <https://go.dev/tour/basics/4>
- Documentation officielle Go, "Function declarations": https://golang.org/ref/spec#Function_declarations
- Go by Example, "Functions": <https://gobyexample.com/functions>
- Go by Example, "Variadic Functions": <https://gobyexample.com/variadic-functions>
- Go Blog, "Error handling and multiple return values": <https://blog.golang.org/error-handling-and-go>

Fonctions anonymes

- **Définition** : Une fonction définie sans nom, souvent directement assignée à une variable ou utilisée comme argument.
- **Syntaxe** :

```
f := func(x int) int {  
    return x * x  
}  
fmt.Println(f(5)) // Affiche 25
```

Cette fonction peut être immédiatement appelée, passée en argument ou retournée.

- **Utilisations courantes** :
 - Passer des fonctions comme arguments (callbacks).
 - Création rapide de fonctions sans nom.
 - Affectation à des variables pour être utilisées plus tard.

Closures en Go : Introduction

- Une **closure** est une fonction anonyme qui **capture les variables** de son environnement lexical.
- Elle peut ainsi accéder et modifier les variables définies dans sa portée englobante, conservant leur état.
- **Exemple simple : un compteur**

```
func compteur() func() int {  
    count := 0 // Variable capturée  
    return func() int {  
        count++  
        return count  
    }  
}  
  
func main() {  
    c := compteur()  
    fmt.Println(c()) // 1  
    fmt.Println(c()) // 2  
    fmt.Println(c()) // 3  
}
```

Explication : La variable `count` est capturée par la fonction anonyme retournée et conserve son état entre les appels.

Closures : Attention aux pièges !

- Il est courant d'avoir des pièges avec les closures capturant des variables dans une boucle.
- **Piège classique : variable partagée**

```
func main() {  
    funcs := []func(){}  
    for i := 0; i < 3; i++ {  
        funcs = append(funcs, func() {  
            fmt.Println(i) // 'i' est partagée par toutes les closures  
        })  
    }  
    for _, f := range funcs {  
        f() // Affiche 3, 3, 3 et non 0,1,2  
    }  
}
```


Cas d'usage pratiques des Closures

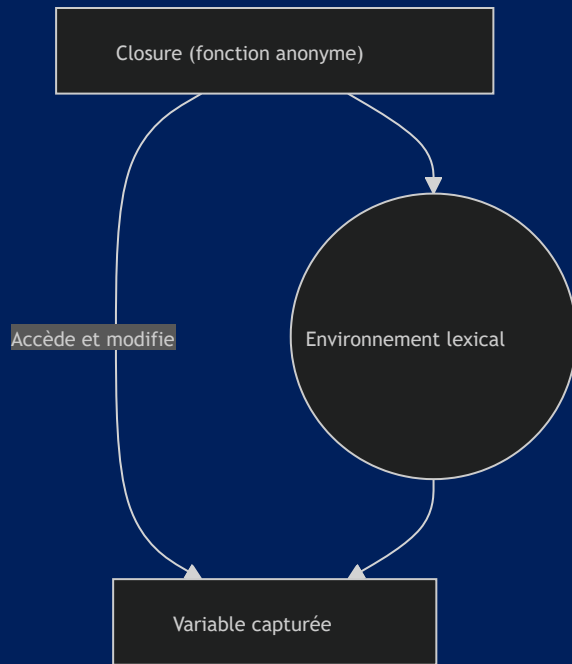
- **a) Fonction génératrice** Les closures permettent de créer des fonctions qui en génèrent d'autres, spécialisées avec un état ou une configuration initiale.

```
func genererMultiplicateur(multiplicateur int) func(int) int {  
    return func(x int) int {  
        return x * multiplicateur  
    }  
}  
  
func main() {  
    doubler := genererMultiplicateur(2)  
    tripler := genererMultiplicateur(3)  
    fmt.Println(doubler(5)) // 10  
    fmt.Println(tripler(5)) // 15  
}
```

- **b) Encapsulation d'état** Elles permettent d'implémenter des structures similaires à des objets légers avec état interne, sans la complexité d'une `struct` Go.

Comprendre la capture d'environnement

Ce diagramme illustre comment une closure capture et interagit avec les variables de son environnement lexical.



Résumé & Ressources

- **Ce qu'il faut retenir**

Concept	Description
Fonction anonyme	Fonction sans nom créée à la volée.
Closure	Fonction anonyme qui capture des variables de son scope.
Capture de variable	Conservation de l'état interne en mémoire.
Pièges	Variables partagées dans les boucles (nécessite une copie locale).

- Ce cours met en lumière comment Go propose des fonctions anonymes puissantes et des closures stockant de l'état, ce qui ouvre la porte à des techniques avancées de programmation fonctionnelle et modulaire.

- **Sources :**

- Documentation officielle Go sur fonctions : https://go.dev/ref/spec#Function_literals
- Tour of Go, "Closures": <https://go.dev/tour/moretypes/23>
- Go by Example, "Closures": <https://gobyexample.com/closures>
- Effective Go, section sur fonctions : https://golang.org/doc/effective_go.html#function_literals

Le type `error` en Go

En Go, la gestion des erreurs est basée sur un type d'interface nommé `error`.

Définition de l'interface `error`

```
type error interface {  
    Error() string  
}
```

Une **erreur** est donc toute valeur qui implémente cette interface, c'est-à-dire qui possède une méthode `Error()` retournant une chaîne descriptive.

Création d'erreurs

a) Utilisation de la fonction `errors.New`

```
import "errors"

err := errors.New("une erreur est survenue")
fmt.Println(err.Error()) // affiche : une erreur est survenue
```

`errors.New` crée une erreur simple avec le message spécifié.

b) Erreurs formatées avec `fmt.Errorf`

```
import "fmt"

err := fmt.Errorf("erreur %d: %s", 404, "ressource non trouvée")
```

Cette fonction permet d'ajouter de la valeur dynamique dans le message.

Indiquer l'absence d'erreur : la valeur `nil`

En Go, l'absence d'erreur est indiquée par la valeur spéciale `nil` (zéro valeur des pointeurs et interfaces).

```
var err error = nil
if err == nil {
    fmt.Println("Pas d'erreur")
} else {
    fmt.Println("Erreur détectée :", err)
}
```

Lorsqu'une fonction retourne une erreur, elle retourne souvent `nil` si tout s'est bien passé, ce qui est la convention à suivre pour indiquer un succès.

Gestion des erreurs : un exemple courant

```
package main

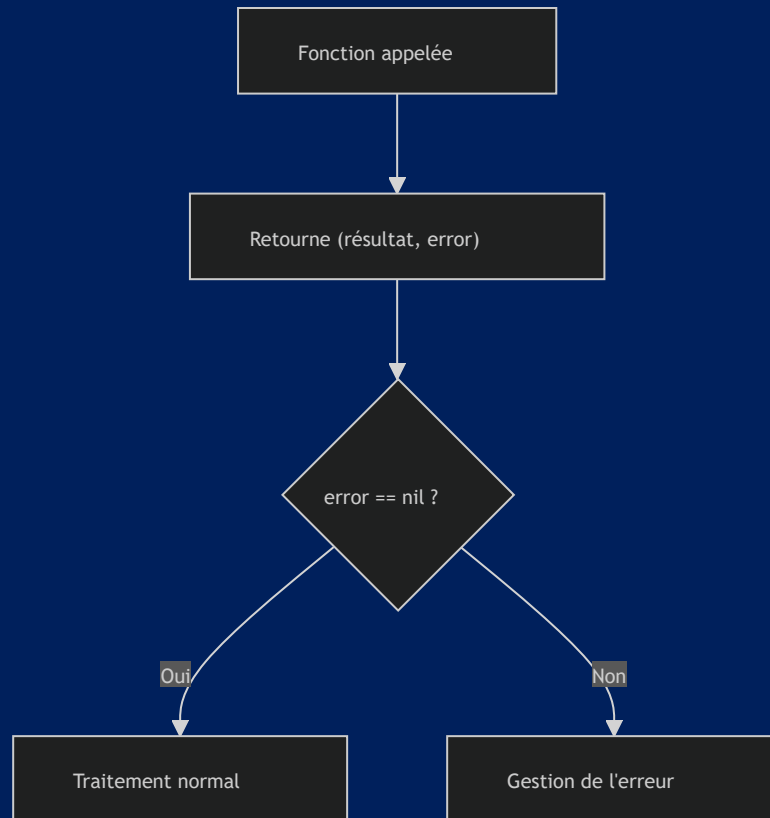
import (
    "errors"
    "fmt"
)

func division(a, b float64) (float64, error) {
    if b == 0 {
        return 0, errors.New("division par zéro interdite")
    }
    return a / b, nil
}

func main() {
    res, err := division(10, 0)
    if err != nil {
        fmt.Println("Erreur :", err)
    } else {
        fmt.Println("Résultat :", res)
    }
}
```

On teste systématiquement si une erreur est présente par comparaison à `nil`.

Cycle de vie d'une erreur en Go



Points clés et ressources

Points clés

Concept	Description
Type <code>error</code>	Interface avec méthode <code>Error() string</code>
Création	<code>errors.New</code> ou <code>fmt.Errorf</code>
Valeur <code>nil</code>	Indique l'absence d'erreur
Vérification	Toujours tester <code>if err != nil</code>
Retour multiple	Souvent <code>(valeur, error)</code>

Sources

- Documentation officielle Go, "Errors": <https://go.dev/doc/tutorial/error-handling>
- Documentation officielle Go, package errors: <https://pkg.go.dev/errors>
- Tour of Go, "Errors": <https://go.dev/tour/moretypes/19>
- Go by Example, "Errors": <https://gobyexample.com/errors>

Gestion des erreurs en Go : `error` et sa création

En Go, le type `error` est une interface prédéfinie :

```
type error interface {  
    Error() string  
}
```

Toute valeur implémentant cette interface peut être utilisée comme erreur.

Pour créer une erreur, on utilise principalement :

- `errors.New` pour les messages simples.
- `fmt.Errorf` pour les messages formatés.

Création d'erreurs simples : `errors.New`

La fonction `errors.New`, du package `errors`, crée une erreur à partir d'une chaîne de caractères. C'est la méthode la plus directe.

```
import "errors"

err := errors.New("une erreur s'est produite")
fmt.Println(err.Error()) // affiche : une erreur s'est produite
```

Création d'erreurs formatées : `fmt.Errorf`

La fonction `fmt.Errorf` permet de formater une chaîne de caractères avant de la transformer en erreur. C'est idéal pour inclure des valeurs dynamiques dans le message d'erreur.

```
import "fmt"

err := fmt.Errorf("erreur n°%d : %s", 404, "ressource non trouvée")
fmt.Println(err) // affiche : erreur n°404 : ressource non trouvée
```

Exemple pratique et le rôle de `nil`

Les fonctions en Go retournent `nil` pour indiquer l'absence d'erreur.

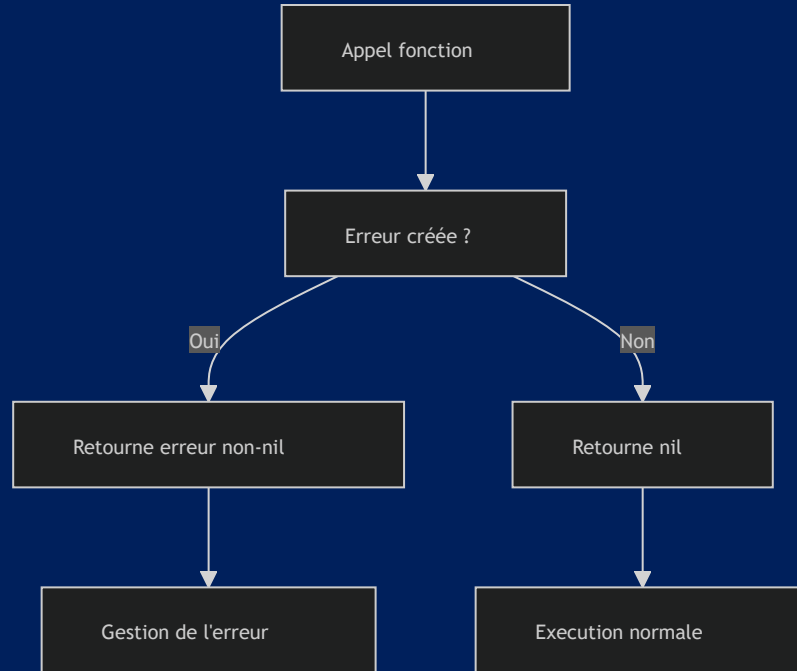
```
package main

import (
    "errors"
    "fmt"
)

func ouvrirFichier(nom string) error {
    if nom == "" {
        return errors.New("nom de fichier vide")
    }
    // Ici, code pour ouvrir le fichier
    return nil // pas d'erreur
}

func main() {
    err := ouvrirFichier("")
    if err != nil { // Comparaison avec nil pour détecter l'erreur
        fmt.Println("Erreur:", err) // Affiche : Erreur: nom de fichier vide
    } else {
        fmt.Println("Fichier ouvert avec succès.")
    }
}
```

Comprendre le flux de gestion d'erreur



L'essentiel à retenir & Ressources

Fonction	Description	Package
<code>errors.New(string)</code>	Crée une erreur simple avec message donné	<code>errors</code>
<code>fmt.Errorf</code>	Crée une erreur avec message formaté	<code>fmt</code>
Valeur <code>nil</code>	Indique absence d'erreur	Langage Go

L'utilisation de `errors.New` et `fmt.Errorf` offre une méthode simple et flexible pour générer et transmettre les erreurs en Go, assurant une communication claire des problèmes rencontrés.

Sources :

- Documentation officielle Go, package errors : <https://pkg.go.dev/errors>
- Go by Example, "Errors" : <https://gobyexample.com/errors>
- Tour of Go, "Errors" : <https://go.dev/tour/moretypes/19>
- Effective Go, partie erreurs : https://golang.org/doc/effective_go#errors

Pourquoi des Erreurs Personnalisées ?

- **Les limites des erreurs standards :**
 - `errors.New` ou `fmt.Errorf` fournissent un simple message texte.
 - Manquent de structure pour transmettre des informations contextuelles.
 - Difficile de distinguer différents types d'erreurs dans le code.
- **Les bénéfices d'une erreur personnalisée (qui implémente `error`) :**
 - **Porte des données additionnelles :** Enrichir l'erreur avec des informations métier.
 - **Catégorise les erreurs par type :** Identifier la nature exacte du problème.
 - **Permet un traitement spécifique :** Adapter la réaction du programme selon le type d'erreur.

Définition : L'Interface `error` et `struct`

- **Principe** : Créer un type `struct` qui implémente la méthode `Error() string`.
- **Exemple** : `ErreurValidation`

```
type ErreurValidation struct {  
    Champ    string  
    Message string  
}  
  
// Implémente la méthode Error() string  
func (e *ErreurValidation) Error() string {  
    return fmt.Sprintf("Erreur dans le champ '%s' : %s", e.Champ, e.Message)  
}
```

Cette `struct` peut contenir des champs pertinents, comme ici `Champ` et `Message`.

Utilisation Basique : Créer et Retourner

- **Retourner une erreur personnalisée depuis une fonction :**

```
func validerNom(nom string) error {  
    if nom == "" {  
        // Création d'une instance de notre erreur personnalisée  
        return &ErreurValidation{  
            Champ:    "Nom",  
            Message:  "ne peut pas être vide",  
        }  
    }  
    return nil  
}
```

- **Vérification simple dans `main` :**

```
func main() {  
    err := validerNom("")  
    if err != nil { // Vérification générique d'erreur  
        fmt.Println("Validation échouée :", err)  
    } else {  
        fmt.Println("Validation réussie")  
    }  
}  
  
// Output: Validation échouée : Erreur dans le champ 'Nom' : ne peut pas être vide
```

La Clé : L'Assertion de Type pour un Traitement Spécifique

- **L'intérêt principal** : Détecter une erreur spécifique pour un traitement dédié.
- **Technique** : Utiliser une **assertion de type**.
- **Exemple d'assertion de type** :

```
// Supposons 'err' est l'erreur retournée par validerNom
if e, ok := err.(*ErreurValidation); ok {
    // 'e' est bien de type *ErreurValidation
    fmt.Printf("Champ en erreur : %s\n", e.Champ)
} else {
    // C'est une autre erreur, ou pas une erreur du tout
    fmt.Println("Autre erreur :", e)
}
```

Ceci permet d'accéder aux champs spécifiques (`Champ` , `Message`) de votre erreur personnalisée.

Scénario Avancé & Visualisation

- Gestion de multiples erreurs avec assertion de type :

```
// validerAge renvoie ErreurValidation si age est hors limites
func validerAge(age int) error { /* ... */ }

erreurs := []error{
    validerAge(-3), // Erreur
    validerAge(34), // OK
    validerAge(200), // Erreur
}

for _, err := range erreurs {
    if err != nil {
        if e, ok := err.(*ErreurValidation); ok {
            fmt.Printf("Validation échouée sur %s : %s\n", e.Champ, e.Message)
        } else {
            fmt.Println("Erreur inconnue :", err)
        }
    } else {
        fmt.Println("Validation OK")
    }
}
```

Points Clés & Ressources

Élément	Détail
Interface <code>error</code>	<code>struct</code> avec méthode <code>Error() string</code> indispensable
Erreur personnalisée	<code>struct</code> avec champs spécifiques + méthode <code>Error()</code>
Assertion de type	Permet de détecter le type concret d'une erreur
Utilisation pratique	Transmet des informations métier, permet des décisions de contrôle fines

Ressources complémentaires :

- Documentation officielle Go, "Errors": <https://go.dev/doc/tutorial/error-handling>
- Go by Example, "Custom Errors": <https://gobyexample.com/custom-error>
- Tour of Go, "Errors": <https://go.dev/tour/moretypes/19>
- Effective Go, "Errors": https://golang.org/doc/effective_go#errors

Cette approche rend la gestion d'erreurs plus fine et robuste, favorisant un code clair et évolutif, avec des erreurs compréhensibles enrichies d'informations supplémentaires.

Les Structures (Structs) en Go : Organisation du Code

Définition & Déclaration d'une `struct`

Les `struct` en Go sont des types composites qui regroupent des champs nommés, chacun pouvant avoir son propre type. Elles permettent de modéliser des entités complexes.

1. Définition d'une `struct`

- Type composite regroupant des champs nommés.
- Les champs peuvent avoir différents types.

2. Syntaxe de déclaration basique

```
type Personne struct {  
    Nom    string // Champ de type string  
    Age    int     // Champ de type int  
    Email  string  // Champ de type string  
}
```

Ici, `Personne` est une structure avec trois champs : `Nom`, `Age`, et `Email`.

Initialisation des `struct` (Partie 1)

Il existe plusieurs façons d'initialiser une `struct` en Go, chacune adaptée à des scénarios spécifiques.

1. Initialisation littérale avec tous les champs (nommés)

- La syntaxe la plus claire et recommandée.
- Nomme explicitement chaque champ.

```
p1 := Personne{
    Nom:    "Dupont",
    Age:    30,
    Email:  "dupont@example.com",
}
```

2. Initialisation littérale avec valeurs positionnelles

- Plus concise, mais moins lisible.
- L'ordre des champs doit impérativement correspondre à la déclaration de la `struct`.

```
p2 := Personne{"Martin", 25, "martin@example.com"}
```

Initialisation des `struct` (Partie 2)

Approches plus spécifiques pour l'initialisation des `struct`.

1. Initialisation partielle

- Seuls certains champs sont spécifiés.
- Les champs non initialisés prennent la **valeur zéro** de leur type (ex: `0` pour `int`, `""` pour `string`, `nil` pour les pointeurs).

```
p3 := Personne{
    Nom: "Leclerc",
}
fmt.Println(p3.Age) // Affiche: 0
```

2. Initialisation avec un constructeur personnalisé

- Pratique courante pour encapsuler la logique d'initialisation.
- Retourne souvent un pointeur vers la nouvelle `struct`.

```
func NouvellePersonne(nom string, age int) *Personne {  
    return &Personne{  
        Nom: nom,  
        Age: age,  
    }  
}  
  
p4 := NouvellePersonne("Dupuis", 40) // p4 est un *Personne
```

Accès, Modification des Champs & Exemple Complet

Une fois initialisée, les champs d'une `struct` sont facilement accessibles et modifiables.

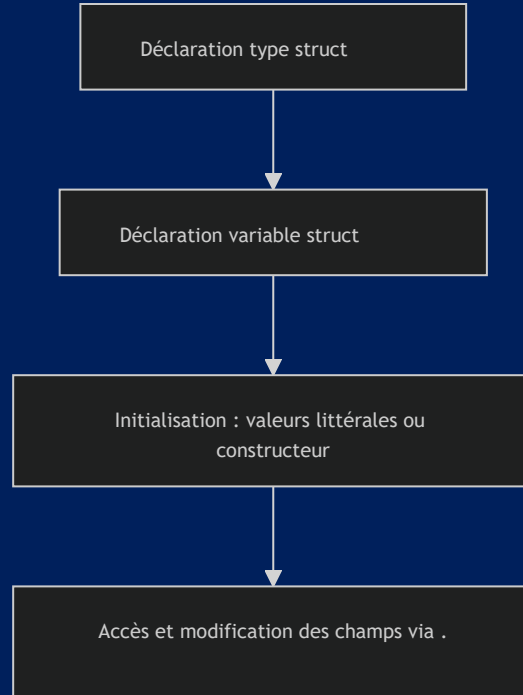
1. Accès et modification des champs

- Utilisation du point (`.`) pour interagir avec les champs.

```
p := Personne{}  
p.Nom = "Lemoine"    // Modification  
p.Age = 28           // Modification  
fmt.Println(p.Nom)   // Accès: Affiche "Lemoine"
```


Cycle de Vie d'une struct

Le diagramme ci-dessous illustre les étapes typiques depuis la définition d'un type `struct` jusqu'à son utilisation.



Points Clés & Ressources

Ce qu'il faut retenir

Élément	Description
<code>type</code> + <code>struct</code>	Crée un nouveau type structuré avec des champs.
Champs nommés	Chaque champ a un nom et un type spécifiques.
Initialisation	Littérale (nommée ou positionnelle) ou via un constructeur.
Valeur zéro	Champs non initialisés prennent la valeur zéro de leur type.
Pointeurs à struct	Fréquemment utilisés pour des références et éviter les copies.

Sources

- Documentation officielle Go, https://go.dev/doc/effective_go#structs
- Tour of Go, "Structs": <https://go.dev/tour/moretypes/5>
- Go by Example, "Structs": <https://gobyexample.com/structs>

Introduction aux méthodes en Go

Les **méthodes** en Go sont des fonctions spéciales qui :

- Sont associées à un **type particulier**, souvent une `struct`.
- Permettent d'attacher un **comportement** aux données, sans utiliser de classes comme en POO.

Déclaration d'une méthode : Une méthode se distingue par un **récepteur** `(r Type)` placé avant son nom :

```
func (p Personne) Bonjour() string {  
    return "Bonjour, je m'appelle " + p.Nom  
}
```

Le récepteur `p Personne` indique que `Bonjour` est une méthode du type `Personne` et peut accéder à ses champs.

Récepteurs en Go : Valeur ou Pointeur ?

Le choix du récepteur impacte la modification des données.

Récepteur valeur (p Personne)

- La méthode reçoit une **copie** de la `struct`.
- Les modifications effectuées à l'intérieur de la méthode **ne sont pas visibles** à l'extérieur.

```
func (p Personne) ChangerNomVal(nom string) {  
    p.Nom = nom // Modifie la copie, pas l'originale  
}
```

Récepteur pointeur (p *Personne)

- La méthode reçoit un **pointeur** vers la `struct` originale.
- Les modifications effectuées à l'intérieur de la méthode **affectent bien la variable d'origine**.

```
func (p *Personne) ChangerNomPtr(nom string) {  
    p.Nom = nom // Modifie l'originale via le pointeur  
}
```


Flexibilité d'appel et types étendus

Appel automatique de méthode

Go simplifie les appels, convertissant automatiquement entre valeur et pointeur si nécessaire :

```
p := Personne{"Alice", 30}
p.ChangerNomPtr("Bob") // Go convertit automatiquement 'p' en '&p' (pointeur)

pPtr := &p
pPtr.Bonjour()          // Go convertit automatiquement 'pPtr' en '*pPtr' (valeur)
```

Méthodes pour tout type nommé

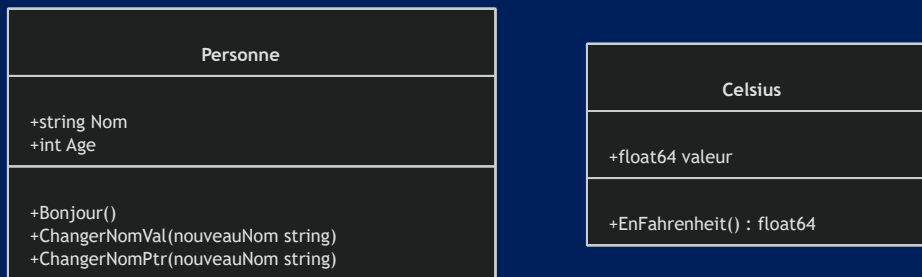
Les méthodes ne sont pas limitées aux `struct` ; elles peuvent être définies pour **tout type nommé** (y compris les alias de types primitifs) :

```
type Celsius float64

func (c Celsius) EnFahrenheit() float64 {
    return float64(c)*1.8 + 32
}
```

Résumé visuel : Structure des méthodes

Un diagramme pour visualiser l'association des méthodes à différents types.



Ce diagramme illustre comment des méthodes spécifiques (*Bonjour* , *ChangerNomVal* , etc.) sont attachées à leurs types respectifs (*Personne* , *Celsius*).

À retenir et ressources complémentaires

Points clés

Élément	Description
Méthode	Fonction associée à un type (avec un récepteur).
Récepteur valeur	Opère sur une copie , modifications locales seulement.
Récepteur pointeur	Opère sur l' originale , permet des modifications persistantes.
Appel simplifié	Go gère les conversions implicites valeur/pointeur.
Méthodes sur types	Utilisable pour <code>struct</code> et autres types nommés .

Sources

- Documentation officielle Go, "Methods": https://go.dev/doc/effective_go#methods
- Tour of Go, "Methods": <https://go.dev/tour/moretypes/17>
- Go by Example, "Methods": <https://gobyexample.com/methods>

Les Pointeurs en Go

Qu'est-ce que c'est et Pourquoi les Utiliser ?

Un **pointeur** est une variable qui stocke l'adresse mémoire d'une autre variable. Il permet d'accéder et modifier directement la valeur stockée à cette adresse, plutôt que de travailler sur une copie.

Utilité essentielle des pointeurs :

- **Performance** : Passer des références aux fonctions (gain de performance, modifications directes).
- **Flexibilité** : Manipuler dynamiquement des structures de données complexes.
- **Méthodes** : Implémenter des méthodes avec récepteurs pointeurs.

Syntaxe de Base des Pointeurs

Déclaration d'un pointeur

```
var p *int // p est un pointeur vers un int (par défaut : nil)
```

Récupérer l'adresse d'une variable (&)

L'opérateur `&` permet d'obtenir l'adresse mémoire d'une variable.

```
x := 42  
p = &x // p pointe vers x (p contient l'adresse de x)
```

Accéder/Modifier la valeur pointée (*)

L'opérateur `*` est utilisé pour déréférencer le pointeur, c'est-à-dire accéder à la valeur à l'adresse qu'il contient.

```
fmt.Println(*p) // affiche 42 (valeur de x via p)  
  
*p = 100        // modifie la valeur de x via p  
fmt.Println(x)  // affiche 100
```

Modifier une variable via un pointeur dans une fonction

Passer un pointeur à une fonction permet à celle-ci de modifier la valeur de la variable d'origine, et non une copie.

```
package main

import "fmt"

func changerValeur(p *int) {
    *p = 10 // Modifie la valeur à l'adresse pointée par p
}

func main() {
    var a int = 5
    fmt.Println("Avant :", a) // Avant : 5

    changerValeur(&a)          // Passe l'adresse de 'a' à la fonction

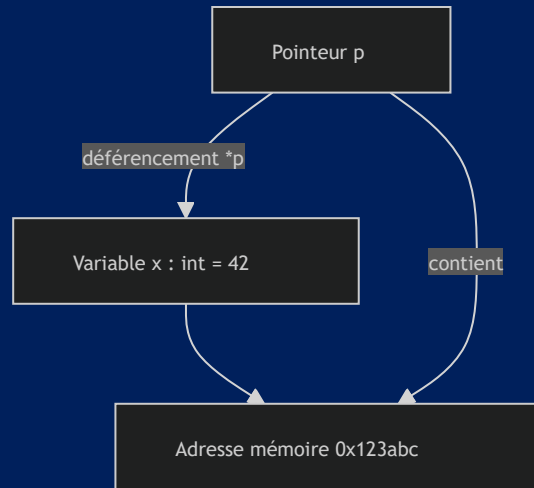
    fmt.Println("Après :", a)  // Après : 10 (la valeur de 'a' a été modifiée)
}
```

Pointeurs et Structures : Efficacité et Manipulation Directe

Les pointeurs évitent la copie complète de structures, souvent volumineuses, et permettent de modifier directement l'instance d'une `struct`.

```
type Personne struct {  
    Nom string  
    Age int  
}  
  
func vieillir(p *Personne) { // Récepteur pointeur  
    p.Age += 1               // Accès et modification directe du champ  
}  
  
func main() {  
    p := Personne{Nom: "Luc", Age: 30}  
    vieillir(&p)             // Passe l'adresse de la struct 'p'  
    fmt.Println(p.Age)       // 31 (l'âge de la struct 'p' a été modifié)  
}
```

Relation variable et pointeur :



Synthèse des Opérateurs et Précautions

Opérateurs clés :

Opérateur	Usage	Exemple
<code>&</code>	Adresse d'une variable	<code>p := &x</code>
<code>*</code>	Dérérencement (valeur pointée)	<code>fmt.Println(*p)</code>

Attention aux pointeurs nuls (`nil`) : Un pointeur non initialisé ou non affecté à une adresse valide a la valeur `nil`. Dérérencer un pointeur `nil` provoque un `panic`. Il est crucial de vérifier sa validité.

```
var p *int = nil
if p != nil {
    fmt.Println(*p) // Cette ligne ne sera pas exécutée
} else {
    fmt.Println("p est nil") // Affiche "p est nil"
}
```

Ce qu'il faut retenir et Ressources

Ce cours introduit les pointeurs en Go avec leur syntaxe et exemples d'usage, permettant une gestion fine et performante des données et facilitant la manipulation d'objets complexes.

Sources pour approfondir :

- Documentation officielle Go, Pointeurs: https://go.dev/doc/effective_go#pointers
- Tour of Go, "Pointers": <https://go.dev/tour/moretypes/6>
- Go by Example, "Pointers": <https://gobyexample.com/pointers>

Passage par Valeur / Référence en Go

Comprendre le Passage par Valeur

En Go, les arguments des fonctions sont par défaut **passés par valeur**. Cela signifie que la fonction reçoit une **copie** de la variable originale.

Une modification effectuée dans la fonction n'affecte donc pas la variable d'origine :

```
func incrementer(x int) {  
    x = x + 1 // Modifie la copie locale de x  
}  
  
a := 5  
incrementer(a)  
fmt.Println(a) // Affiche 5 (l'original n'a pas été modifié)
```

Besoin de modifier l'original ? Il faut utiliser le passage par référence avec un pointeur.

Passage par Référence en Go

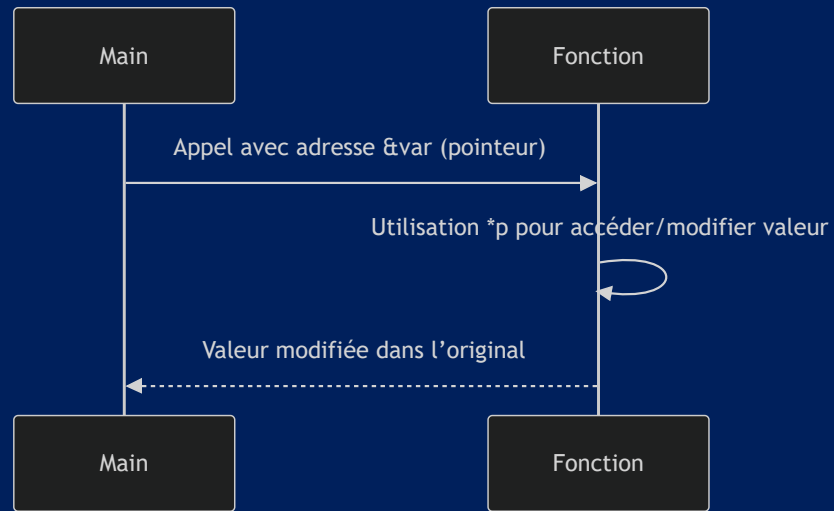
Maîtriser le Passage par Référence avec les Pointeurs

Pour modifier la valeur d'origine, on passe **l'adresse mémoire** de la variable avec l'opérateur `&` (opérateur d'adresse).

```
func incrementerPtr(x *int) { // La fonction reçoit un pointeur vers un entier
    *x = *x + 1               // Déférérencement (*x) pour accéder et modifier la valeur réelle
}

a := 5
incrementerPtr(&a)           // Passe l'adresse de 'a'
fmt.Println(a)               // Affiche 6 (la valeur originale a été modifiée)
```

Mécanisme visuel du passage par référence :



Passage par Référence en Go

Avantages du Passage par Référence

L'utilisation des pointeurs pour le passage par référence offre plusieurs avantages significatifs :

- **Permet la modification de la variable d'origine** même à l'intérieur d'une fonction.
- **Évite des copies coûteuses** de grandes structures ou de tableaux, améliorant les performances.
- **Largement utilisé** avec des `structs` ou des objets plus volumineux pour une gestion efficace de la mémoire.

Passage par Référence en Go

Application Concrète : Modifier une Structure (Struct)

Le passage par pointeur est particulièrement utile avec les `structs` pour permettre leur modification directe.

```
type Point struct {  
    X, Y int  
}  
  
func deplacer(p *Point, dx, dy int) { // La fonction reçoit un pointeur vers un Point  
    p.X += dx                          // Accès direct aux champs de la struct via le pointeur  
    p.Y += dy  
}  
  
func main() {  
    pt := Point{X: 1, Y: 2}           // Création d'une instance de Point  
    deplacer(&pt, 3, 4)               // Passe l'adresse de 'pt'  
    fmt.Println(pt)                   // Affiche {4 6} - la struct originale est modifiée  
}
```

Passage par Référence en Go

Comparaison : Struct par Valeur vs par Pointeur

Il est crucial de comprendre la différence d'impact entre passer une `struct` par valeur ou par pointeur.

Passage par Valeur d'une Struct : Une copie de la struct est passée. Les modifications locales n'affectent pas l'original.

```
func changerX(p Point) { // Reçoit une copie de 'Point'
    p.X = 100
}

pt := Point{10, 20}
changerX(pt)
fmt.Println(pt.X) // Affiche 10. La modification locale de 'p.X' n'a pas affecté 'pt.X'.
```

Passage par Pointeur d'une Struct : L'adresse de la struct est passée. Les modifications affectent directement l'original.

```
func changerXPointeur(p *Point) { // Reçoit un pointeur vers 'Point'
    p.X = 100
}

changerXPointeur(&pt) // Passe l'adresse de 'pt'
fmt.Println(pt.X)     // Affiche 100. La modification est effective sur 'pt.X'.
```

Passage par Référence en Go

Points d'Attention & Ressources Clés

Ce qu'il faut retenir : Le passage par référence via pointeurs en Go est fondamental pour modifier directement les variables passées en argument et optimiser la manipulation des données volumineuses.

Attention au passage par référence :

- Le pointeur doit être **valide (non nil)** pour éviter des erreurs (panic) à l'exécution.
- La manipulation de pointeurs doit rester **claire et mesurée** pour ne pas compliquer inutilement la lecture et la maintenance du code.

Sources pour aller plus loin :

- Documentation officielle Go, paramétrage et pointeurs : go.dev/doc/effective_go#passing
- Tour of Go, "Pointers": go.dev/tour/moretypes/6
- Go by Example, "Pointers": gobyexample.com/pointers

L'essentiel des Packages Go

- **Qu'est-ce qu'un package ?**

- Unité de compilation et d'organisation du code source en Go.
- Regroupe un ensemble de fichiers `.go` partageant un même espace de noms.
- Déclaration obligatoire en début de fichier : `package monpackage`.
- Le package central de l'exécutable est toujours `package main`.

- **Que contient un package ?**

- Des fonctions, des types (structs, interfaces), des variables et constantes, des méthodes associées.
- **Visibilité** : Seuls les identifiants **exportés** (commençant par une majuscule) sont accessibles depuis l'extérieur du package.

Créer un Package Go

- **Principe**

- Chaque package est un répertoire distinct.
- Le nom du package correspond généralement au nom du répertoire.

- **Exemple : Package** `mathutil`

```
/monprojet  
  /mathutil  
    mathutil.go  
    main.go
```

- **Contenu de** `mathutil/mathutil.go`

```
package mathutil  
  
func Square(x int) int { // Fonction exportée  
    return x * x  
}
```

La fonction `Square` commence par une majuscule pour être accessible depuis l'extérieur du package.

Utiliser un Package Go

- **Importation**

- La directive `import` permet d'inclure les packages nécessaires.
- Le chemin d'importation indique où trouver le package.

- **Exemple d'utilisation dans** `main.go`

```
package main

import (
    "fmt"
    "monprojet/mathutil" // Import du package utilisateur
)

func main() {
    // Accès à la fonction Square exportée du package mathutil
    fmt.Println(mathutil.Square(5)) // Affiche 25
}
```

- Le package est importé avec `import "monprojet/mathutil"`.
- L'identifiant `Square` est accessible car il commence par une majuscule.

Organisation et Types de Packages

- **Organisation dans le Workspace**

- Chaque package est un répertoire distinct.
- Le nom des packages correspond généralement au nom du répertoire.
- Le point d'entrée `package main` contient la fonction `main()`.
- Pour les projets plus complexes, les packages sont organisés en sous-répertoires.

- **Packages Standards vs Utilisateurs**

- **Packages Standards** : Go fournit une bibliothèque standard riche accessible via l'import.
 - Exemples : `"fmt"`, `"net/http"`, `"encoding/json"`.
- **Packages Utilisateurs** : Peuvent être importés via des modules Go (via `go.mod`) en précisant leur chemin complet.

Exemple Avancé et Structure Visuelle

- **Export de Type et Méthode dans un package**

```
package shapes // Package 'shapes'

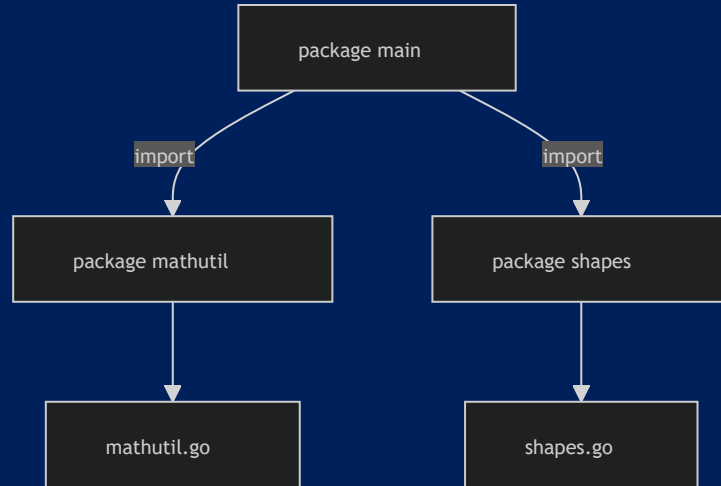
type Rectangle struct { // Type 'Rectangle' exporté
    Width, Height int
}

func (r Rectangle) Area() int { // Méthode 'Area' exportée
    return r.Width * r.Height
}
```

Utilisation dans un autre package :

```
// ... import "monprojet/shapes"
r := shapes.Rectangle{Width: 5, Height: 3}
fmt.Println(r.Area()) // Affiche 15
```

- Organisation Basique des Packages



Synthèse et Ressources

- **Points à Retenir**

Élément	Description
<code>package</code>	Directive obligatoire en tête du fichier
Visibilité/exportation	Identifiants avec majuscule accessibles partout
Structure	Un package = un répertoire
Importation	<code>import "chemin/du/package"</code>
Packages standard & tiers	Disponibles via import et modules Go

- **Sources**

- Documentation officielle Go, Packages : `https://go.dev/doc/code`
- Effective Go, packages : `https://golang.org/doc/effective\_go#package-names`
- Go by Example, Packages : `https://gobyexample.com/packages`

Introduction à la structure d'un projet Go

Un projet Go s'organise autour d'une convention simple et standardisée, essentielle pour :

- La maintenance
- La lisibilité
- Le déploiement

Depuis l'arrivée des **modules Go** (`go mod`), la gestion des dépendances et la structure sont devenues plus **flexibles** tout en respectant les bonnes pratiques. Cette organisation favorise la clarté et la collaboration.

Les Fondations : `go.mod` et `cmd/`

1. Le fichier `go.mod`

- **Définit le module** (nom du projet ou chemin d'import).
- Gère les **dépendances et versions** via Go Modules.

```
module github.com/monuser/monprojet

go 1.20

require (
    github.com/some/dependency v1.2.3
)
```

2. Le dossier `cmd/`

- Contient les **applications principales** (binaries).
- Chaque sous-dossier représente une application avec un `main.go`.
- Permet d'avoir **plusieurs commandes** dans un même dépôt.

```
cmd/
  app1/
    main.go
  app2/
    main.go
```

L'Organisation du Code Métier

1. Les packages métiers dans `pkg/`

- Contient le **code réutilisable**, organisé par domaine fonctionnel.
- Exemple : `pkg/service/`, `pkg/dao/`, `pkg/utils/`

2. Le dossier `internal/`

- Les packages sont **accessibles uniquement au sein du dépôt**.
- Restriction imposée par le compilateur Go.
- Protège le code non destiné à être utilisé en dehors du projet.
- Exemple : `internal/authentication/`, `internal/cache/`

3. Le dossier `api/` (optionnel)

- Contient les contrats API (fichiers swagger, protobuf, etc.).

Structure Typique d'un Projet Go

Voici un exemple minimal et son usage type des fichiers :

Structure du projet

```
monprojet/  
├── go.mod  
├── cmd/  
│   └── serveur/  
│       └── main.go  
├── pkg/  
│   ├── calcul/  
│   │   └── calcul.go  
│   └── util/  
│       └── util.go  
└── internal/  
    ├── db/  
    │   └── conn.go
```

Exemple de `go.mod`

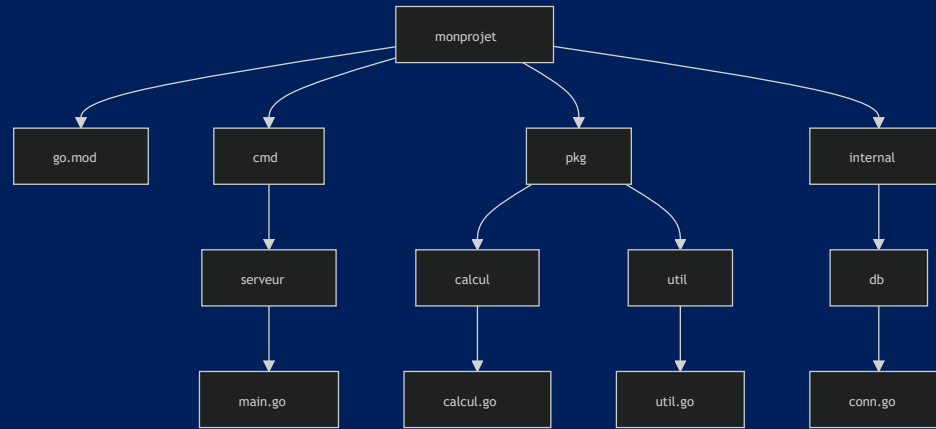
```
module github.com/monuser/monprojet

go 1.20

require github.com/stretchr/testify v1.8.2
```

Usage typique des fichiers

- `cmd/serveur/main.go` : Point d'entrée, initialise le serveur.
- `pkg/calcul/calcul.go` : Fonctions métier (ex: calculs).
- `internal/db/conn.go` : Gestion base de données, code non exposé hors module.



Bonnes Pratiques pour un Projet Go

Élément	Recommandation
Modules	Toujours initialiser avec <code>go mod init</code>
Dossier <code>cmd/</code>	Contient tous les points d'entrée
Dossier <code>pkg/</code>	Fonctionnalités exportables et réutilisables
Dossier <code>internal/</code>	Code privé au module, éviter les fuites d'abstraction
Nommage	Garder noms courts, parlants, cohérents

Ce qu'il faut retenir & Ressources Utiles

Cette structure standard, basée sur la modularité et la clarté, est essentielle pour la **maintenance**, la **collaboration** et la **compréhension** de vos projets Go.

Ressources complémentaires :

- Documentation officielle Go : <https://go.dev/doc/modules>
- Guide Go Project Layout : <https://github.com/golang-standards/project-layout>
- Blog Go : Organizing Go Programs <https://go.dev/doc/code#Organization>

1. Que fait `internal/` exactement ?

Le dossier `internal/` est une **convention officielle du langage Go** (voir [doc Go officielle](#)).

→ **But** : limiter l'accès à certains packages à l'intérieur du module.

Concrètement :

- Tout code **à l'extérieur** du module ne peut **pas importer** de packages situés dans `internal/`.
- Par contre, **les autres packages du même module** (et ses sous-dossiers) **peuvent importer** `internal/`.

Exemple

```
monprojet/  
├─ internal/db/conn.go  
├─ pkg/api/api.go  
└─ main.go
```

✓ `pkg/api/api.go` peut faire :

```
import "monprojet/internal/db"
```

✗ Mais un autre projet externe ne peut pas faire :

```
import "github.com/monpseudo/monprojet/internal/db"
```

→ Cela provoquera une **erreur de compilation** :

```
use of internal package not allowed
```

2. Ce que `internal/` ne fait pas

Il **ne sécurise pas** les données ou le code :

- Il **ne chiffre pas** le contenu.
- Il **n'empêche pas l'accès physique** aux fichiers.
- Il **ne protège pas les secrets** (ex. mot de passe DB, token API...).

C'est simplement une **barrière de compilation logique** :

Un garde-fou contre les imports non voulus, pas contre les intrusions.

3. Comment protéger réellement les **secrets** (base de données, clés API, etc.)

On gère ça **à l'exécution**, pas dans le code.

Voici les **bonnes pratiques** pour Go :

a. Variables d'environnement

C'est la méthode la plus courante et recommandée.

Exemple :

```
# Linux / macOS
export DB_USER="root"
export DB_PASS="s3cr3t"
export DB_NAME="mydb"
```


b. Fichier `.env` (avec `github.com/joho/godotenv`)

`.env` :

```
DB_USER=root  
DB_PASS=secret  
DB_NAME=mydb
```


c. Gestion centralisée via un *secret manager*

En production, utilise un vrai service de gestion de secrets :

- **AWS Secrets Manager**
- **GCP Secret Manager**
- **Vault (HashiCorp)**
- **Docker secrets / Kubernetes secrets**

Ces systèmes permettent :

- de **chiffrer les secrets au repos** ;
- de **les injecter dynamiquement à l'exécution** ;
- et d'**éviter qu'ils apparaissent dans le code source**.

d. Paramètres chiffrés ou fichiers de configuration

Une autre approche (moins courante mais valable) : charger un fichier `config.json` chiffré et le déchiffrer en mémoire au démarrage. Mais cette méthode nécessite une **clé de déchiffrement stockée ailleurs**, donc elle revient souvent à la même problématique.

4. En résumé

Élément	But principal	Sécurité réelle ?	Exemple
<code>internal/</code>	Empêcher les imports externes	✗ Non	<code>import "monprojet/internal/db"</code>
Variables d'environnement	Stocker les secrets hors du code	✓ Oui	<code>os.Getenv("DB_PASS")</code>
<code>.env</code> + <code>godotenv</code>	Facilité en dev local	⚠ Partielle (jamais en prod)	<code>.env</code> non versionné
Secret Manager	Gestion centralisée et chiffrée	✓ ✓ Haute	AWS, GCP, Vault...

Structure du projet

```
monprojet/  
├── go.mod  
├── cmd/  
│   └── serveur/  
│       └── main.go  
├── internal/  
│   └── db/  
│       └── conn.go  
└── .env
```

Fichier `.env` (*non versionné !*)

```
DB_USER=root  
DB_PASS=supersecret  
DB_HOST=localhost  
DB_PORT=3306  
DB_NAME=todoapp
```

Et dans ton `.gitignore` :

```
.env
```

internal/db/conn.go

```
package db

import (
    "database/sql"
    "fmt"
    "log"
    "os"
    "time"

    _ "github.com/go-sql-driver/mysql" // pilote MySQL
    "github.com/joho/godotenv"
)
```


cmd/serveur/main.go

```
package main

import (
    "log"
    "net/http"
    "monprojet/internal/db"
)

func main() {
    // Initialiser la BDD
    db.Init()
    defer db.Close()

    // Exemple : serveur basique
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        w.Write([]byte("Hello depuis le serveur Go"))
    })

    log.Println("🌐 Serveur lancé sur :8080")
    if err := http.ListenAndServe(":8080", nil); err != nil {
        log.Fatal(err)
    }
}
```

Explication (bonne pratique)

Élément	Objectif	Détails
<code>godotenv.Load()</code>	Charger les variables d'environnement locales	Pratique en dev, inutile (et ignoré) en prod
<code>os.Getenv(...)</code>	Récupérer les secrets sans les exposer dans le code	Sécurité : rien n'est hardcodé
<code>sql.Open(...)</code>	Initialise un <i>pool</i> de connexions, ne connecte pas encore	Utilise le DSN formaté avec les valeurs du <code>.env</code>
<code>db.Ping()</code>	Vérifie qu'une vraie connexion à la base fonctionne	Bonne pratique avant de lancer le serveur
<code>SetMaxOpenConns , SetMaxIdleConns</code>	Contrôle de la charge du pool de connexions	Évite les fuites de connexions
<code>defer db.Close()</code>	Fermeture propre à la fin du programme	Empêche les leaks
<code>log.Fatal</code> / <code>log.Println</code>	Gestion claire des erreurs	Affiche immédiatement les problèmes de config ou réseau

En résumé

- `internal/` empêche d'importer `db` depuis un autre projet.
- Les secrets restent **hors du code source** (dans `.env` ou variables système).
- La connexion est testée, gérée, et proprement fermée.
- Le code est “**production-friendly**”, **lisible**, et **idiomatique Go**.

Go et le Web: Consommation d'API avec `net/http`

Le package `net/http`: Votre allié pour les API

Le package `net/http` est la bibliothèque standard de Go pour gérer les protocoles HTTP. Il permet de créer des serveurs HTTP et de réaliser des requêtes clients.

Pour la consommation d'API (clients HTTP):

- `http.Client` : Pour un contrôle avancé des requêtes.
- `http.Get` : Pour des requêtes GET.
- `http.Post` : Pour des requêtes POST.

Effectuer une requête GET et lire la réponse

La fonction `http.Get` permet d'envoyer une requête GET vers une URL et récupérer la réponse.

```
resp, err := http.Get("https://api.example.com/data")
if err != nil {
    log.Fatal(err)
}
defer resp.Body.Close() // Toujours fermer le corps de la réponse

body, err := io.ReadAll(resp.Body) // Lecture du corps
if err != nil {
    log.Fatal(err)
}

fmt.Println(string(body)) // Affichage de la réponse
```

- `resp.Body` est un `io.ReadCloser` qui doit être fermé pour libérer les ressources.
- Le corps de la réponse est lu avec `io.ReadAll`.

Récupération et décodage JSON

Les API renvoient souvent du JSON qu'il faut parser avec `encoding/json`.

```
package main

import (
    "encoding/json"
    "fmt"
    "io"
    "log"
    "net/http"
    "time"
)

// Exemple de structure correspondant à la réponse JSON
type User struct {
    ID      int    `json:"id"`
    Name    string `json:"name"`
    Email   string `json:"email"`
}
```

```
func main() {  
    // 1 Création du client avec timeout global  
    client := &http.Client{  
        Timeout: 10 * time.Second,  
    }  
  
    // 2 Création de la requête GET  
    req, err := http.NewRequest("GET", "https://jsonplaceholder.typicode.com/users/1", nil)  
    if err != nil {  
        log.Fatalf("Erreur de création de la requête : %v", err)  
    }  
  
    // 3 (Optionnel) ajout d'en-têtes  
    req.Header.Set("Accept", "application/json")  
}
```

```
// 4 Exécution de la requête
resp, err := client.Do(req)
if err != nil {
    log.Fatalf("Erreur d'exécution de la requête : %v", err)
}
defer resp.Body.Close() // Toujours fermer le corps

// 5 Vérification du code HTTP
if resp.StatusCode != http.StatusOK {
    body, _ := io.ReadAll(resp.Body)
    log.Fatalf("Erreur HTTP %d : %s", resp.StatusCode, string(body))
}

// 6 Lecture du corps et décodage JSON
var user User
if err := json.NewDecoder(resp.Body).Decode(&user); err != nil {
    log.Fatalf("Erreur de décodage JSON : %v", err)
}

// 7 Affichage du résultat
fmt.Printf("✅ Requête réussie (%s)\n", resp.Status)
fmt.Printf("Utilisateur : %+v\n", user)
}
```

Étape	Code / Élément	Rôle / Explication	Bonnes pratiques appliquées
1	<pre>client := &http.Client{ Timeout: 10 * time.Second }</pre>	Crée un client HTTP réutilisable avec un timeout global de 10 s pour éviter de bloquer indéfiniment.	✅ Timeout obligatoire en production
2	<pre>http.NewRequest("GET", url, nil)</pre>	Prépare une requête GET. On passe nil car il n'y a pas de corps pour un GET.	✅ Permet d'ajouter facilement des en-têtes ensuite
3	<pre>req.Header.Set("Accept", "application/json")</pre>	Indique au serveur que le client souhaite du JSON.	✅ Clarifie le type de réponse attendu
4	<pre>resp, err := client.Do(req)</pre>	Exécute la requête. Retourne la réponse et l'erreur réseau éventuelle.	✅ <code>client.Do()</code> donne plus de contrôle que <code>http.Get()</code>
4b	<pre>defer resp.Body.Close()</pre>	Assure la fermeture du corps à la fin de main , même en cas d'erreur ultérieure.	✅ Prévention de fuite de connexion
5	<pre>if resp.StatusCode != http.StatusOK { ... }</pre>	Vérifie que le serveur a renvoyé un code 200. En cas d'erreur, on lit le corps pour logger le message.	✅ Gestion explicite des codes 4xx/5xx
6	<pre>json.NewDecoder(resp.Body).Decode(&user)</pre>	Décode directement le flux JSON en structure Go.	✅ Plus économe en mémoire, idiomatique
7	<pre>fmt.Printf("✅ Requête réussie ... ")</pre>	Affiche le statut et la structure user avec ses champs.	✅ Lisibilité et retour clair pour le dev

Envoi d'une requête POST avec un corps JSON

```
package main

import (
    "bytes"
    "encoding/json"
    "fmt"
    "io"
    "log"
    "net/http"
    "time"
)

// Struct représentant le corps à envoyer en JSON
type Payload struct {
    Name string `json:"name"`
    Age  int    `json:"age"`
}
```

--> Suite -->

```
func main() {  
    // 1 Données à envoyer  
    data := Payload{Name: "Alice", Age: 30}  
  
    // 2 Encodage JSON (directement dans un buffer)  
    var buf bytes.Buffer  
    if err := json.NewEncoder(&buf).Encode(data); err != nil {  
        log.Fatalf("Erreur d'encodage JSON : %v", err)  
    }  
  
    // 3 Création d'un client HTTP avec timeout  
    client := &http.Client{  
        Timeout: 10 * time.Second, // limite la durée totale de la requête  
    }  
  
    // 4 Création de la requête POST  
    req, err := http.NewRequest("POST", "https://api.example.com/users", &buf)  
    if err != nil {  
        log.Fatalf("Erreur de création de la requête : %v", err)  
    }  
}
```

--> Suite -->

```
// Ajout de l'en-tête Content-Type
req.Header.Set("Content-Type", "application/json")

// 5 Envoi de la requête
resp, err := client.Do(req)
if err != nil {
    log.Fatalf("Erreur d'envoi de la requête : %v", err)
}
defer resp.Body.Close() // toujours fermer le corps, même en cas d'erreur HTTP

// 6 Vérification du code de statut HTTP
if resp.StatusCode < 200 || resp.StatusCode ≥ 300 {
    body, _ := io.ReadAll(resp.Body)
    log.Fatalf("Erreur HTTP %d : %s", resp.StatusCode, string(body))
}

// 7 Lecture du corps de la réponse
body, err := io.ReadAll(resp.Body)
if err != nil {
    log.Fatalf("Erreur de lecture du corps : %v", err)
}

// 8 Traitement de la réponse
fmt.Printf("✅ Requête réussie (%s)\n", resp.Status)
fmt.Println("Réponse :", string(body))
}
```

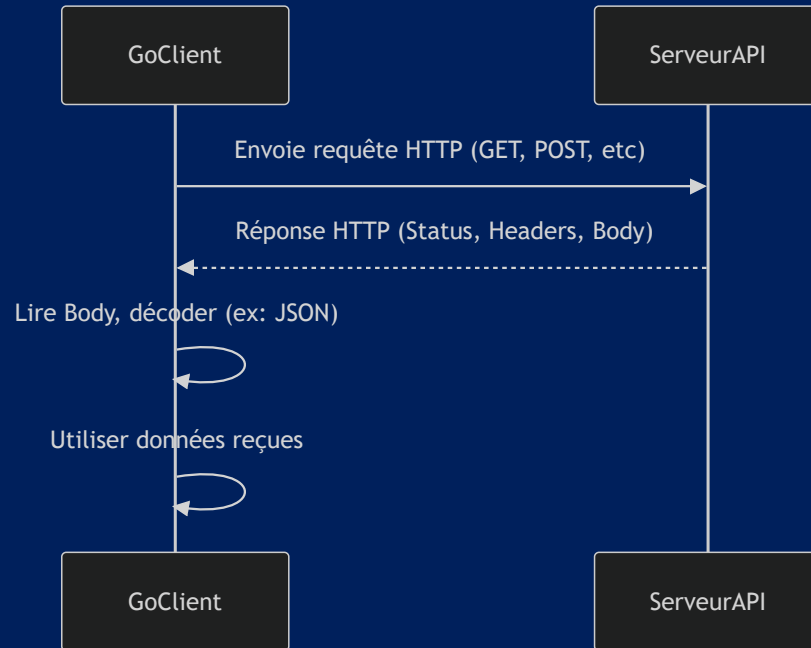
Étape	Description
1.	On crée la structure <code>Payload</code> à envoyer.
2.	On encode directement le JSON dans un <code>bytes.Buffer</code> , évitant ainsi <code>json.Marshal</code> + <code>bytes.NewBuffer</code> .
3.	On utilise un <code>http.Client</code> avec un timeout global (meilleure pratique en production).
4.	On crée la requête via <code>http.NewRequest</code> pour pouvoir ajouter des en-têtes facilement.
5.	<code>client.Do(req)</code> exécute la requête (au lieu de <code>http.Post</code>).
6.	On vérifie le code HTTP — les codes 4xx/5xx sont considérés comme des erreurs, donc on les log avec le corps de la réponse pour débogage.
7.	On lit le corps avec <code>io.ReadAll</code> et on le ferme proprement.
8.	On affiche le statut et le contenu de la réponse.

Gestion des erreurs et cycle de vie d'une requête

Bonnes pratiques

- Toujours vérifier la variable `err` après chaque appel HTTP.
- Vérifier `resp.StatusCode` (200-299 = succès).
- Gérer proprement la fermeture de `resp.Body` avec `defer`.

Flux de requête HTTP client



Sources utiles

- Documentation `net/http` officielle : <https://pkg.go.dev/net/http>
- Go by Example, HTTP Clients : <https://gobyexample.com/http-requests>

Go et le Web : Consommation d'API

Requêtes HTTP : Récupérer et Envoyer des Données

- **Requête GET** : Utilisée pour **récupérer des ressources** (ex: données JSON, fichiers) depuis un serveur.
- **Requête POST** : Utilisée pour **envoyer des données** au serveur, par exemple pour créer une nouvelle ressource.

Requête HTTP GET en Go : Récupérer des Données

La requête GET sert à **récupérer des ressources** depuis un serveur.

Méthode la plus simple :

```
resp, err := http.Get(url)
```

Requête POST Avancée et Flux de Requête HTTP

Passage de headers personnalisés avec `http.Client` : Pour un contrôle plus fin (headers, méthodes non standard), utilisez `http.NewRequest` .

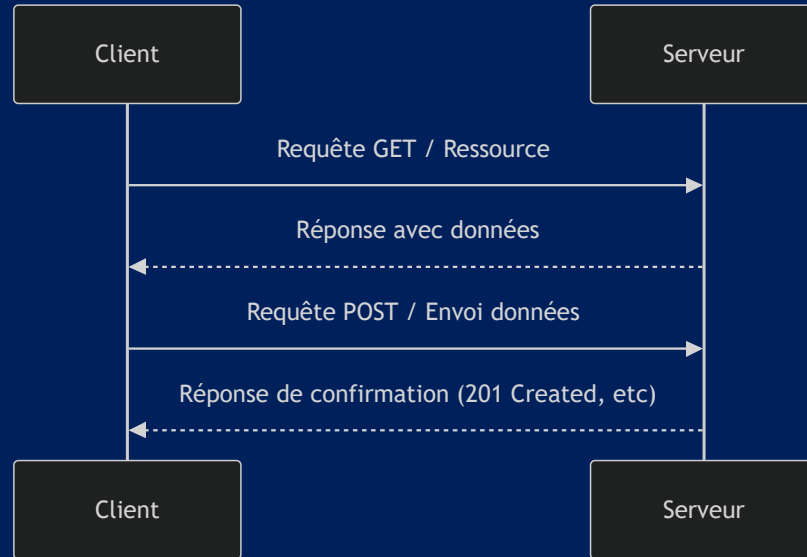
```
client := &http.Client{}

// Création manuelle de la requête POST
req, err := http.NewRequest("POST", url, bytes.NewBuffer(jsonValue))
if err != nil {
    log.Fatal(err)
}

// Ajout de headers personnalisés
req.Header.Set("Content-Type", "application/json")
req.Header.Set("Authorization", "Bearer mytoken")

// Exécution de la requête
resp, err := client.Do(req)
if err != nil {
    log.Fatal(err)
}
defer resp.Body.Close()
```

Flux d'une requête HTTP :



GET vs POST : Comparaison Essentielle

Critère	GET	POST
Usage	Récupération de données	Envoi/modification de données
Corps de requête	Généralement vide	Contient les données à transmettre
Idempotence	Oui (répété, même effet)	Non nécessairement idempotente
Sécurité	Paramètres dans URL (limites de taille)	Données dans corps (plus sûr)

Bonnes Pratiques et Ressources Utiles

Conseils pratiques :

- Toujours **fermer le corps de réponse** avec `defer resp.Body.Close()` pour éviter les fuites mémoire.
- Vérifier le **code HTTP** (status) avant de traiter la réponse.
- Pour POST, s'assurer d'encoder proprement les données (souvent en JSON).
- Pour des headers spécifiques ou méthodes HTTP non standard, utiliser `http.NewRequest`.

Ce qu'il faut retenir : Ce cours montre comment effectuer des requêtes HTTP GET et POST en Go, avec des exemples pour récupérer et envoyer des données JSON, ainsi que les bonnes pratiques d'usage du package standard `net/http`.

Sources :

- Documentation `net/http` : <https://pkg.go.dev/net/http>
- Go by Example – HTTP clients : <https://gobyexample.com/http-requests>
- JSON Placeholder (API test simple) : <https://jsonplaceholder.typicode.com/>

Go et le Web : Marshalling et Unmarshalling JSON

Traitement JSON en Go

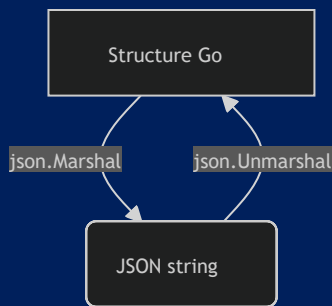
1. Introduction & Concepts Clés

JSON (JavaScript Object Notation) est un format de données texte léger, très utilisé pour l'échange d'informations (APIs).

Go offre un support natif pour la conversion entre structures Go et JSON via le package `encoding/json`.

2. Marshalling & Unmarshalling : Le Cycle

Terme	Description
Marshalling	Conversion d'une structure Go en JSON (serialization)
Unmarshalling	Conversion de JSON en structure Go (désérialisation)



Marshalling : De Go vers JSON

```
package main

import (
    "encoding/json"
    "fmt"
    "log"
)

type User struct {
    ID    int    `json:"id"`
    Name  string `json:"name"`
    Email string `json:"email,omitempty"` // omit empty champ si vide
}

func main() {
    user := User{ID: 1, Name: "Alice"}
    jsonData, err := json.Marshal(user)
    if err != nil {
        log.Fatal(err)
    }
    fmt.Println(string(jsonData)) // {"id":1,"name":"Alice"}
}
```

- Les tags struct (`json:"id"`) indiquent le nom du champ dans JSON.
- `omitempty` omet le champ si sa valeur est vide.

Unmarshalling : De JSON vers Go

```
package main

import (
    "encoding/json"
    "fmt"
    "log"
)

type User struct {
    ID    int    `json:"id"`
    Name  string `json:"name"`
    Email string `json:"email"`
}

func main() {
    jsonStr := `{"id":42,"name":"Bob","email":"bob@example.com"}`
    var user User
    if err := json.Unmarshal([]byte(jsonStr), &user); err != nil {
        log.Fatal(err)
    }
    fmt.Printf("User : %+v\n", user) // User : {ID:42 Name:Bob Email:bob@example.com}
}
```

- Il est nécessaire de passer un **pointeur** à `Unmarshal` pour modifier la variable.
- Les champs JSON doivent correspondre aux tags ou noms dans la struct.

Cas Avancés et Robustesse

Types génériques

Pour des JSON complexes ou inconnus, utiliser des `map[string]interface{}` ou des slices :

```
var data map[string]interface{}
jsonStr := `{"name":"Eve","age":29,"skills":["Go","Python"]}`

if err := json.Unmarshal([]byte(jsonStr), &data); err != nil {
    log.Fatal(err)
}

fmt.Println(data["skills"]) // [Go Python]
```

Gestion des erreurs

Le marshal ou unmarshal peut échouer. **Toujours traiter l'erreur** :

```
err := json.Unmarshal([]byte(badJSON), &var)
if err != nil {
    // gestion erreur
}
```

Synthèse et Ressources

Points importants

Aspect	Détail
Tags JSON	Contrôle la représentation JSON
Pointeurs pour Unmarshal	Nécessaires pour modifier les variables
Types compatibles	Structures, maps, slices, primitives
<code>omitempty</code>	Omet les champs vides lors du marshallage

En conclusion

Ce cours présente les mécanismes fondamentaux du traitement JSON en Go, avec les opérations de marshalling et unmarshalling, illustrées par des exemples simples et orientés pratique.

Sources

- Documentation officielle `encoding/json` : <https://pkg.go.dev/encoding/json>
- Go by Example, JSON : <https://gobyexample.com/json>
- Tour of Go, JSON : <https://go.dev/tour/moretypes/23>

Développement d'API REST avec Go

1. Introduction & Rappels sur REST

Une API **REST** (*Representational State Transfer*) est un style d'architecture logicielle permettant à des systèmes hétérogènes de communiquer via le protocole HTTP.

Les principes clés :

- **Ressources** : Chaque élément manipulé est identifié par une URL unique (ex: `/api/v1/users/42`).
- **Verbes HTTP** :
 - **GET** : Récupérer une ressource ou une liste de ressources.
 - **POST** : Créer une nouvelle ressource.
 - **PUT** : Remplacer/mettre à jour intégralement une ressource.
 - **PATCH** : Mettre à jour partiellement une ressource.
 - **DELETE** : Supprimer une ressource.
- **Format de données** : Le format standard utilisé pour les échanges est le **JSON** (`application/json`).
- **Stateless (Sans état)** : Chaque requête contient toutes les informations nécessaires à son traitement.

2. Pourquoi Go pour le développement d'API ?

Go (Golang) a été conçu par Google pour répondre aux problématiques d'infrastructures modernes et de systèmes distribués.

- **Performance & Compilation** : Fichier binaire unique, rapide à démarrer, consommation mémoire minime.
- **Concurrence native** : Les *goroutines* permettent de gérer des milliers de requêtes simultanées à moindre coût.
- **Bibliothèque standard puissante** : Le package `net/http` est prêt pour la production sans dépendance externe obligatoire.
- **Typage fort et simplicité** : Code lisible, maintenable et moins sujet aux erreurs d'exécution.

3. Fondations : Le package standard `net/http`

En Go, il n'est pas obligatoire d'installer un framework externe (comme Express en JS ou Spring en Java) pour créer un serveur HTTP. Le package natif `net/http` fournit tout le nécessaire.

Exemple minimaliste : Serveur HTTP "Hello World"

```
package main

import (
    "fmt"
    "net/http"
)

func main() {
    // Association d'un chemin à une fonction de traitement
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintf(w, "Bienvenue sur notre API en Go !")
    })

    // Démarrage du serveur sur le port 8080
    fmt.Println("Serveur démarré sur http://localhost:8080")
    err := http.ListenAndServe(":8080", nil)
    if err != nil {
        panic(err)
    }
}
```

4. Comprendre et écrire des Handlers

4.1. L'interface `http.Handler`

Tout composant capable de traiter une requête HTTP en Go implémente l'interface native :

```
type Handler interface {  
    ServeHTTP(ResponseWriter, *Request)  
}
```

Les deux paramètres fondamentaux sont :

1. `http.ResponseWriter` : Permet de construire la réponse HTTP (définir les headers, le status code et le corps).
2. `*http.Request` : Contient toutes les données de la requête entrante (URL, méthode, headers, paramètres de requête, body).

4.2. `http.HandlerFunc`

Pour éviter de devoir créer une `struct` pour chaque handler, Go fournit le type adapter `http.HandlerFunc`. Il convertit une simple fonction ayant la signature `func(http.ResponseWriter, *http.Request)` en un `http.Handler`.

4.3. Manipulation du JSON (Sérialisation / Désérialisation)

Dans une API REST, la réception et l'envoi de données se font principalement en JSON via le package `encoding/json`.

```
package main

import (
    "encoding/json"
    "net/http"
)

type Article struct {
    ID      int    `json:"id"`
    Titre   string `json:"titre"`
    Auteur  string `json:"auteur"`
}

func GetArticleHandler(w http.ResponseWriter, r *http.Request) {
    // 1. Définir le header Content-Type
    w.Header().Set("Content-Type", "application/json")

    // 2. Créer une instance de struct
    art := Article{
        ID:      1,
        Titre:   "Développer en Go",
        Auteur:  "Alice",
    }

    // 3. Encoder en JSON et envoyer la réponse
    w.WriteHeader(http.StatusOK) // Code 200
    json.NewEncoder(w).Encode(art)
}
```

--> Suite

```
func CreateArticleHandler(w http.ResponseWriter, r *http.Request) {  
    w.Header().Set("Content-Type", "application/json")  
  
    var newArt Article  
  
    // Décoder le JSON reçu dans le body  
    err := json.NewDecoder(r.Body).Decode(&newArt)  
    if err != nil {  
        http.Error(w, "Format JSON invalide", http.StatusBadRequest) // Code 400  
        return  
    }  
  
    newArt.ID = 42 // Simulation de la génération d'un ID  
  
    // Renvoyer la ressource créée avec un statut 201 Created  
    w.WriteHeader(http.StatusCreated)  
    json.NewEncoder(w).Encode(newArt)  
}
```

5. Le Routage (Routing) en Go

Le routage consiste à diriger une requête HTTP vers le bon handler selon l'URL et la méthode HTTP.

5.1. Le ServeMux standard (Go 1.22+)

Depuis Go 1.22, le multiplexeur standard `http.ServeMux` supporte nativement :

- La spécification des verbes HTTP dans le pattern (`GET /items` , `POST /items`).
- Les paramètres dynamiques d'URL (ex: `/items/{id}`).

```
func main() {
    mux := http.NewServeMux()

    // Routage par méthode et chemin dynamique
    mux.HandleFunc("GET /articles", GetArticles)
    mux.HandleFunc("GET /articles/{id}", GetArticleByID)
    mux.HandleFunc("POST /articles", CreateArticle)

    http.ListenAndServe(":8080", mux)
}

func GetArticleByID(w http.ResponseWriter, r *http.Request) {
    // Extraction du paramètre d'URL {id}
    id := r.PathValue("id")
    w.Write([]byte("Détails de l'article ID : " + id))
}
```

5.2. Utilisation d'un routeur tiers (ex: `go-chi/chi`)

Dans les projets professionnels ou les anciennes versions de Go, on utilise souvent un routeur tiers léger et compatible avec `net/http` comme `chi` :

```
r := chi.NewRouter()
r.Get("/articles/{id}", GetArticleByID)
```

6. Les Middlewares en Go

Un **middleware** est un composant logiciel qui intercepte la requête avant qu'elle n'atteigne le handler principal (ou après son exécution).

Cas d'usage typiques :

- Logging des requêtes.
- Authentification & Autorisation (ex: vérification de Token JWT).
- Gestion des erreurs globales (Panic Recovery).
- Gestion du CORS (*Cross-Origin Resource Sharing*).

Pattern d'un Middleware en Go

En Go, un middleware est une fonction qui prend un `http.Handler` en paramètre et retourne un nouveau `http.Handler`.

Middleware : `http.Handler` $\longrightarrow$ `http.Handler`

```
func LoggingMiddleware(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
        // --- Action AVANT le handler principal ---  
        log.Printf("[%s] %s", r.Method, r.URL.Path)  
  
        // Appel du handler suivant dans la chaîne  
        next.ServeHTTP(w, r)  
  
        // --- Action APRES le handler principal ---  
        log.Println("Requête traitée avec succès")  
    })  
}
```

Exemple pratique : Chainage et Authentification

```
func AuthMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        token := r.Header.Get("Authorization")

        if token != "Bearer secret123" {
            http.Error(w, "Non autorisé", http.StatusUnauthorized)
            return // Interruption de la chaîne
        }

        // Continuer si l'authentification est valide
        next.ServeHTTP(w, r)
    })
}
```

Checklist des Bonnes Pratiques REST en Go :

1. **Gestion explicite des erreurs** : Toujours vérifier les erreurs renvoyées par `json.NewDecoder` et `w.Write`.
2. **Codes HTTP adéquats** :
 1. `200 OK` (Succès générique)
 2. `201 Created` (Ressource créée)
 3. `204 No Content` (Suppression réussie)
 4. `400 Bad Request` (Données fournies invalides)
 5. `401 Unauthorized` / `403 Forbidden` (Problème de droits)
 6. `404 Not Found` (Ressource inexistante)
 7. `500 Internal Server Error` (Erreur serveur)

Sources :

- Documentation officielle Go `net/http` : <https://pkg.go.dev/net/http>
- Go Web Examples : <https://gowebexamples.com/gin/>

Développement d'API REST avec Go

2. Exposition de services – Framework Gin

Introduction

Gin est un framework web léger et performant pour Go, spécialement conçu pour faciliter et accélérer le développement d'API REST. Il fournit une API simple, un routage rapide, un middleware flexible et de nombreuses fonctionnalités utiles comme la gestion des requêtes JSON, la validation, et la gestion des erreurs, tout en conservant la simplicité du package standard `net/http`.

1. Pourquoi utiliser Gin ?

- **Performance élevée** : Gin est parmi les frameworks les plus rapides en Go grâce à sa conception efficace.
- **Routage expressif** : Supporte les routes avec paramètres, groupes, routes imbriquées.
- **Middleware efficace** : Possibilité d'intercaler des middlewares à tous les niveaux — global, groupe, route.
- **Support JSON natif** : Encodage/décodage facile des données.
- **Gestion intégrée des erreurs, validation, récupération des paniques.**
- **Documentation et communauté actives.**

2. Installation

```
go get -u github.com/gin-gonic/gin
```

3. Premier exemple avec Gin

```
package main

import (
    "github.com/gin-gonic/gin"
    "net/http"
)

func main() {
    r := gin.Default() // crée un routeur avec middleware Logger et Recovery

    r.GET("/ping", func(c *gin.Context) {
        c.JSON(http.StatusOK, gin.H{"message": "pong"})
    })

    r.Run(":8080") // démarre le serveur sur le port 8080
}
```

4. Routage et handlers

Gin permet d'extraire facilement les paramètres d'URL et d'affecter des handlers à différentes méthodes HTTP :

```
r.GET("/user/:id", func(c *gin.Context) {  
    id := c.Param("id")  
    c.JSON(http.StatusOK, gin.H{"user_id": id})  
})
```

Pour les paramètres de requête (query strings) :

```
r.GET("/search", func(c *gin.Context) {  
    query := c.Query("q")  
    page := c.DefaultQuery("page", "1")  
    c.JSON(http.StatusOK, gin.H{"query": query, "page": page})  
})
```

5. Middleware dans Gin

Gin utilise les middlewares basés sur le même principe de chaînage que `net/http`. Exemple de middleware simple :

```
func MyMiddleware() gin.HandlerFunc {  
    return func(c *gin.Context) {  
        // Log avant la requête  
        fmt.Println("Début de la requête")  
        c.Next() // passe au handler suivant  
        // Log après la requête  
        fmt.Println("Fin de la requête")  
    }  
}  
  
func main() {  
    r := gin.Default()  
  
    r.Use(MyMiddleware())  
  
    r.GET("/ping", func(c *gin.Context) {  
        c.JSON(200, gin.H{"message": "pong"})  
    })  
  
    r.Run()  
}
```

6. Exemple complet : API CRUD simple avec Gin

```
package main

import (
    "github.com/gin-gonic/gin"
    "net/http"
    "strconv"
)

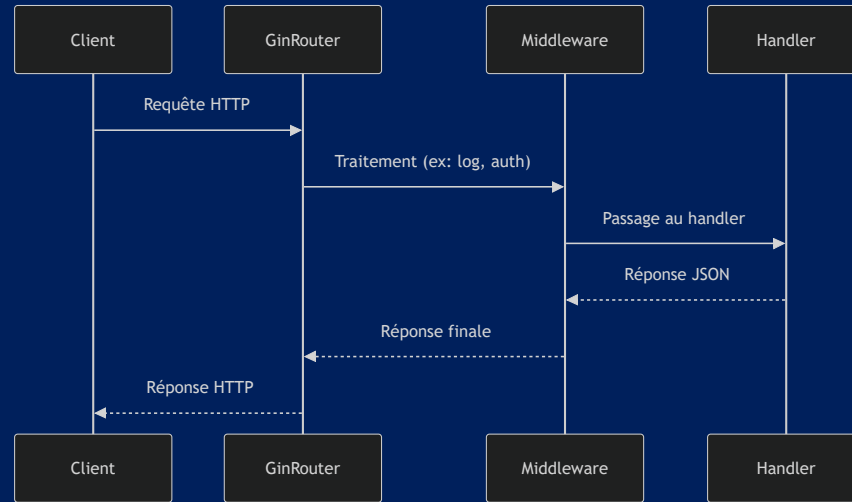
type User struct {
    ID int `json:"id"`
    Name string `json:"name"`
}

var users = []User{{ID: 1, Name: "Alice"}, {ID: 2, Name: "Bob"}}
```

--> Suite

```
func main() {  
    r := gin.Default()  
  
    r.GET("/users", func(c *gin.Context) {  
        c.JSON(http.StatusOK, users)  
    })  
  
    r.GET("/users/:id", func(c *gin.Context) {  
        id, _ := strconv.Atoi(c.Param("id"))  
        for _, u := range users {  
            if u.ID == id {  
                c.JSON(http.StatusOK, u)  
                return  
            }  
        }  
        c.JSON(http.StatusNotFound, gin.H{"message": "Utilisateur non trouvé"})  
    })  
  
    r.POST("/users", func(c *gin.Context) {  
        var newUser User  
        if err := c.ShouldBindJSON(&newUser); err != nil {  
            c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})  
            return  
        }  
        newUser.ID = len(users) + 1  
        users = append(users, newUser)  
        c.JSON(http.StatusCreated, newUser)  
    })  
}
```

7. Diagramme Mermaid – Flux simple d'une requête dans Gin



8. Sources

- [Gin Web Framework - Documentation officielle](#)
- [Gin by Example](#)
- [Go Web Examples - Gin](#)
- [Awesome Go - Web Frameworks](#)

1. Introduction & Prérequis

En Go, la gestion des bases de données relationnelles repose sur une interface d'abstraction fournie par le langage : la bibliothèque standard `database/sql`.

Cependant, selon la complexité et l'architecture de votre projet, trois approches principales existent :

1. `database/sql` : Le package standard bas niveau (performant, explicite, mais verbeux).
2. `sqlx` : Une surcouche légère de `database/sql` facilitant le mapping vers des `structs`.
3. `GORM` : Un ORM (*Object-Relational Mapping*) complet apportant un haut niveau d'abstraction.

2. Partie 1 : Le Package Standard `database/sql`

2.1 Principaux concepts

Le package `database/sql` fournit une interface générique. Pour l'utiliser avec un SGBD spécifique, il faut importer anonymement le **driver** correspondant (via `import _ "driver/path"`).

En Go, le type `sql.DB` ne représente **pas** une seule connexion physique à la base, mais un **pool de connexions** géré automatiquement par le runtime.

2.2 Connexion et driver SQLite

Pour SQLite, on utilise couramment un driver sans dépendance CGO comme modernc.org/sqlite ou github.com/glebarez/go-sqlite .

```
package main

import (
    "database/sql"
    "fmt"
    "log"

    _ "modernc.org/sqlite" // Import anonyme du driver SQLite
)

func main() {
    // Connexion (ou création du fichier database.db s'il n'existe pas)
    db, err := sql.Open("sqlite", "app.db")
    if err != nil {
        log.Fatalf("Erreur d'ouverture de la BD :", err)
    }
    defer db.Close()

    // Vérification de la disponibilité de la connexion
    if err := db.Ping(); err != nil {
        log.Fatalf("Impossible de joindre la base de données :", err)
    }
}
```

2.3 Exécution de requêtes de modification (Exec)

Pour les instructions qui ne renvoient pas de lignes d'enregistrement (CREATE TABLE , INSERT , UPDATE , DELETE), on utilise la méthode

`db.Exec()` .

```
func createTable(db *sql.DB) error {
    query := `
    CREATE TABLE IF NOT EXISTS etudiants (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nom TEXT NOT NULL,
        email TEXT UNIQUE NOT NULL,
        age INTEGER
    );`

    _, err := db.Exec(query)
    return err
}
```


2.4 Lecture de données (Query et QueryRow)

A. Lecture d'un seul enregistrement (QueryRow)

Lorsqu'une requête doit retourner **au plus une ligne**, on utilise `db.QueryRow()` .

```
type Etudiant struct {
    ID    int
    Nom   string
    Email string
    Age   int
}

func getEtudiantByID(db *sql.DB, id int) (*Etudiant, error) {
    query := `SELECT id, nom, email, age FROM etudiants WHERE id = ?`

    row := db.QueryRow(query, id)

    var e Etudiant
    err := row.Scan(&e.ID, &e.Nom, &e.Email, &e.Age)
    if err != nil {
        if err == sql.ErrNoRows {
            return nil, fmt.Errorf("aucun étudiant trouvé avec l'id %d", id)
        }
        return nil, err
    }

    return &e, nil
}
```

B. Lecture de plusieurs enregistrements (Query)

Lorsque la requête retourne **plusieurs lignes**, on utilise `db.Query()`. Il est **obligatoire** de libérer les ressources avec `defer rows.Close()`.

```
func getAllEtudiants(db *sql.DB) ([]Etudiant, error) {
    query := `SELECT id, nom, email, age FROM etudiants`

    rows, err := db.Query(query)
    if err != nil {
        return nil, err
    }
    defer rows.Close() // Toujours fermer le curseur !

    var etudiants []Etudiant
    for rows.Next() {
        var e Etudiant
        if err := rows.Scan(&e.ID, &e.Nom, &e.Email, &e.Age); err != nil {
            return nil, err
        }
        etudiants = append(etudiants, e)
    }
    // Vérification d'éventuelles erreurs survenues pendant la boucle
    if err := rows.Err(); err != nil {
        return nil, err
    }
    return etudiants, nil
}
```

2.5 Prévention des injections SQL

Règle d'or : Ne formulez *jamais* vos requêtes SQL en concaténant des chaînes de caractères avec des entrées utilisateurs.

✗ A NE PAS FAIRE (Vulnérable aux injections SQL) :

```
// DANGER !  
query := fmt.Sprintf("SELECT * FROM etudiants WHERE nom = '%s'", nomSaisi)
```

✓ A FAIRE (Requêtes préparées / Paramétrées) :

```
// Sécurisé : le driver échappe automatiquement les paramètres.  
query := "SELECT * FROM etudiants WHERE nom = ?"  
rows, err := db.Query(query, nomSaisi)
```

2.6 Transactions (`sql.Tx`)

Les transactions garantissent les propriétés **ACID** (Atomicité, Cohérence, Isolation, Durabilité). En Go, une transaction s'initialise avec `db.Begin()`.

```
func transfertEtudiant(db *sql.DB, etudiantID int) error {
    tx, err := db.Begin()
    if err != nil {
        return err
    }

    // En cas de panique ou d'erreur non gérée, le Defer effectuera un Rollback
    // Si la transaction a été Commit(), Rollback() n'aura aucun effet.
    defer tx.Rollback()
    // Étape 1 : Insertion de l'historique
    _, err = tx.Exec("INSERT INTO archive_etudiants (etudiant_id) VALUES (?)", etudiantID)
    if err != nil {
        return err // Déclenche le defer tx.Rollback()
    }
    // Étape 2 : Suppression de la table active
    _, err = tx.Exec("DELETE FROM etudiants WHERE id = ?", etudiantID)
    if err != nil {
        return err // Déclenche le defer tx.Rollback()
    }
    // Validation de la transaction
    return tx.Commit()
}
```

3. Partie 2 : Extension avec `sqlx`

3.1 Pourquoi utiliser `sqlx` ?

Le package `github.com/jmoiron/sqlx` est une extension directe de `database/sql`. Il ne masque pas le SQL, mais élimine la répétition manuelle des instructions `Scan()` grâce au **struct mapping** automatique.

3.2 Structure et tags db

`sqlx` fait la liaison entre les colonnes de la base et les champs de la structure via les tags `db:"nom_colonne"`.

```
package main

import (
    "log"

    "github.com/jmoiron/sqlx"
    _ "modernc.org/sqlite"
)

type Etudiant struct {
    ID      int    `db:"id"`
    Nom     string `db:"nom"`
    Email   string `db:"email"`
    Age     int    `db:"age"`
}
```

3.3 Lecture simplifiée (Get et Select)

- `db.Get()` : Remplace `QueryRow()` + `Scan()` pour un seul élément.
- `db.Select()` : Remplace `Query()` + la boucle `rows.Next()` + `Scan()` pour une tranche (*slice*).

```
func demoSqlx(db *sqlx.DB) {  
    // 1. Connexion avec sqlx  
    // db, err := sqlx.Open("sqlite", "app.db")  
  
    // 2. Récupérer un seul enregistrement  
    var e Etudiant  
    err := db.Get(&e, "SELECT id, nom, email, age FROM etudiants WHERE id = ?", 1)  
    if err != nil {  
        log.Println("Erreur Get :", err)  
    }  
  
    // 3. Récupérer une liste complète  
    var etudiants []Etudiant  
    err = db.Select(&etudiants, "SELECT * FROM etudiants WHERE age ≥ ?", 18)  
    if err != nil {  
        log.Println("Erreur Select :", err)  
    }  
}
```

3.4 Requêtes nommées (NamedExec)

`sqlx` permet d'associer directement les variables nommées aux champs d'une structure :

```
func insertEtudiantNamed(db *sqlx.DB, e Etudiant) error {  
    query := `INSERT INTO etudiants (nom, email, age) VALUES (:nom, :email, :age)`  
  
    _, err := db.NamedExec(query, e)  
    return err  
}
```

4. Partie 3 : Utilisation d'un ORM avec GORM

4.1 Concept d'ORM

Un **ORM** (*Object-Relational Mapper*) abstrait la couche de base de données en associant automatiquement des objets (structures Go) à des tables SQL.

Vous n'écrivez plus de requêtes SQL brutes pour les opérations courantes.

4.2 Définition des structures et Auto-migration

GORM fournit une structure de base `gorm.Model` contenant les champs standard (`ID` , `CreatedAt` , `UpdatedAt` , `DeletedAt`).

```
package main

import (
    "fmt"
    "log"

    "glebarez/go-sqlite" // Driver SQLite compatible pure Go pour GORM
    "gorm.io/gorm"
)

type Etudiant struct {
    gorm.Model          // Ajoute automatiquement ID, CreatedAt, UpdatedAt, DeletedAt
    Nom                 string `gorm:"size:100;not null"`
    Email               string `gorm:"uniqueIndex;not null"`
    Age                 int
}
```


4.3 Opérations CRUD

A. Création (Create)

```
e := Etudiant{Nom: "Alice", Email: "alice@example.com", Age: 20}  
result := db.Create(&e)  
  
// result.Error contient l'erreur éventuelle  
// result.RowsAffected renvoie le nombre de lignes insérées  
fmt.Println("Nouvel ID :", e.ID)
```

B. Lecture (Read)

```
var e Etudiant

// Trouver le premier enregistrement
db.First(&e, 1) // SELECT * FROM etudiants WHERE id = 1 LIMIT 1

// Filtrer avec condition
var etudiants []Etudiant
db.Where("age ≥ ?", 18).Find(&etudiants)
```

C. Mise à jour (Update)

```
var e Etudiant
db.First(&e, 1)

// Mettre à jour un seul champ
db.Model(&e).Update("Nom", "Alice Modified")

// Mettre à jour plusieurs champs avec Map ou Struct
db.Model(&e).Updates(Etudiant{Nom: "Alice Dupont", Age: 21})
```

D. Suppression (Delete)

```
var e Etudiant
db.First(&e, 1)

// Suppression logique (Soft Delete car gorm.Model est utilisé)
db.Delete(&e)

// Pour forcer une suppression définitive (Hard Delete) :
// db.Unscoped().Delete(&e)
```

4.4 Aperçu des associations (1-N)

GORM permet de gérer très facilement les relations inter-tables.

```
type Promotion struct {
    gorm.Model
    Nom      string
    Etudiants []Etudiant `gorm:"foreignKey:PromotionID"` // Relation 1 à N
}

type Etudiant struct {
    gorm.Model
    Nom      string
    Email    string
    PromotionID uint // Clé étrangère
}

func getPromotionAvecEtudiants(db *gorm.DB, promoID uint) (*Promotion, error) {
    var promo Promotion
    // Preload charge automatiquement les étudiants associés (jointure transparente)
    err := db.Preload("Etudiants").First(&promo, promoID).Error
    if err != nil {
        return nil, err
    }
    return &promo, nil
}
```

5. Tableau Comparatif & Synthèse

Critère	database/sql	sqlx	GORM
Niveau d'abstraction	Bas niveau (Contrôle total)	Intermédiaire	Haut niveau (ORM)
Performances	Max (Aucune surcouche)	Quasi-identiques à standard	Équivalentes/Légèrement inférieures
Mapping Struct	Manuel via <code>Scan()</code>	Automatique via <code>Select</code> / <code>Get</code>	Automatique
Ligne de code / Verbosité	Élevée	Modérée	Faible
Courbe d'apprentissage	Faible (si SQL maîtrisé)	Très faible	Moyenne
Cas d'usage recommandé	Microservices légers, besoins de perf stricte	Projets backend standards avec requêtes SQL maîtrisées	Applications d'entreprise complexes, CRUD rapide

Découvrir la concurrence en Go

Goroutines et Channels

1. Introduction à la concurrence en Go

La concurrence en Go est native et intégrée au langage via deux mécanismes majeurs :

- **Les goroutines** : exécution concurrente légère des fonctions.
- **Les channels** : mécanisme de communication et de synchronisation entre goroutines.

2. Goroutines : Exécution Concurrente Légère

Une **goroutine** est une fonction ou méthode qui s'exécute de manière concurrente, lancée avec l'instruction `go`.

- Léger coût mémoire (~kilobytes) contrairement aux threads système.
- Le runtime Go gère leur planification (scheduler M:N).
- Permet d'exploiter efficacement les multi-cœurs.

```
func sayHello() { /* ... */ }

func main() {
    go sayHello() // Lancement concurrent
    // ...
}
```

Sans `go`, `sayHello()` s'exécuterait avant la suite du `main`.

3. Channels : Communication et Synchronisation

Les **channels** permettent la communication sécurisée entre goroutines : passer des valeurs, synchroniser des actions.

- Déclaré avec `make(chan Type)`.
- Syntaxe d'envoi : `ch ← value`
- Syntaxe de réception : `value := ← ch`
- Bloquent l'émetteur si le canal est plein, et le récepteur si vide (synchronisation).

```
func sum(nums []int, result chan int) {  
    // ... calcul total  
    result ← total // Envoi du résultat  
}  
  
func main() {  
    result := make(chan int)  
    go sum(nums, result)  
    val := ←result // Réception du résultat  
    // ...  
}
```

4. Channels Bufferisés et Sélection Multiple

Channels Bufferisés

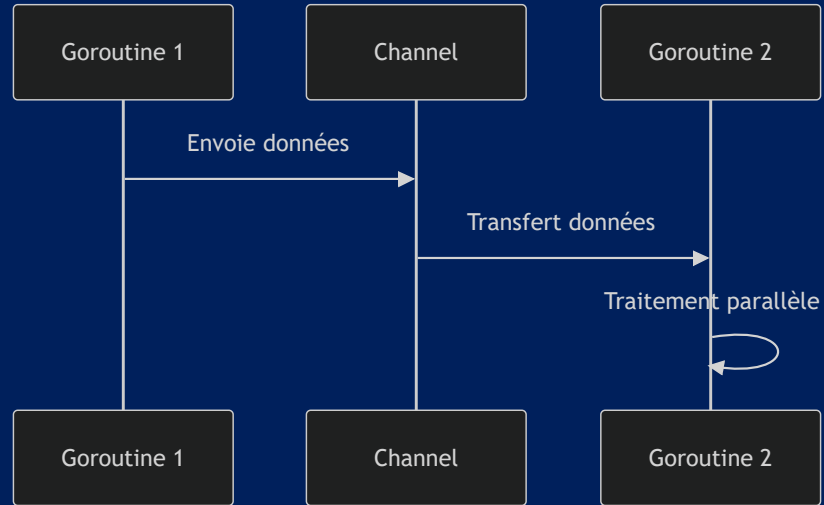
- `make(chan Type, taille)` : tampon de capacité fixe.
- Non bloquant tant que le buffer n'est pas plein.

Sélection avec `select`

Permet d'écouter plusieurs channels simultanément et d'agir sur le premier canal prêt.

```
select {
case val := <-ch1:
    fmt.Println("Reçu depuis ch1", val)
case ch2 <- 42:
    fmt.Println("Envoyé 42 sur ch2")
default:
    fmt.Println("Aucun channel prêt")
}
```

5. Comprendre la Communication



6. Points Clés & Ressources

Ce qu'il faut retenir

Élément	Description
Goroutines	Exécution concurrente légère (fonction)
Channels	Passage de messages, synchronisation
Blocage	Channels synchronisent producteurs & consommateurs
Buffer	Permet découpler producteurs et consommateurs
<code>select</code>	Permet gérer plusieurs opérations en concurrence

Sources

- Documentation officielle Go - Goroutines : https://go.dev/doc/effective_go#goroutines
- Documentation `channel` : https://go.dev/doc/effective_go#channels
- Go by Example - Goroutines et Channels : <https://gobyexample.com/concurrency>

Découvrir les perspectives de Go : Synchronisation basique

Concurrence en Go : Pourquoi synchroniser ?

La concurrence en Go s'appuie sur l'exécution simultanée de **goroutines**. Lorsqu'elles partagent des ressources (variables, structures), il devient nécessaire de **synchroniser** l'accès pour :

- Éviter des conditions de course.
- Garantir l'intégrité des données.

`sync.Mutex` : Protection des accès concurrents

Le package `sync` fournit le type `Mutex` (verrou) pour protéger une section critique :

- Seule une goroutine peut verrouiller un mutex à la fois.
- Les autres goroutines attendent la libération pour accéder à la section protégée.

`sync.WaitGroup` : Coordonner la fin des goroutines

`WaitGroup` permet de **synchroniser la fin d'un ensemble de goroutines**.

- `Add(n)` : Indique le nombre de goroutines à attendre.
- Chaque goroutine appelle `Done()` à sa fin.
- Le `Wait()` bloque jusqu'à ce que le compteur atteigne zéro.

Cet outil assure la coordination et la fin propre du programme.

Synchronisation avancée et application combinée

Autres primitives de synchronisation :

- `sync.RWMutex` : Mutex optimisé lecture/écriture (plusieurs lecteurs, un seul écrivain).
- `sync.Once` : Exécution unique d'un bloc de code même en concurrence.
- `sync.Cond` : Condition variable pour synchroniser selon un état spécifique.

```
package main

import (
    "fmt"
    "sync"
)

var (
    balance int
    mu      sync.Mutex
)

func deposit(amount int, wg *sync.WaitGroup) {
    defer wg.Done()
    mu.Lock()
    balance += amount
    mu.Unlock()
}

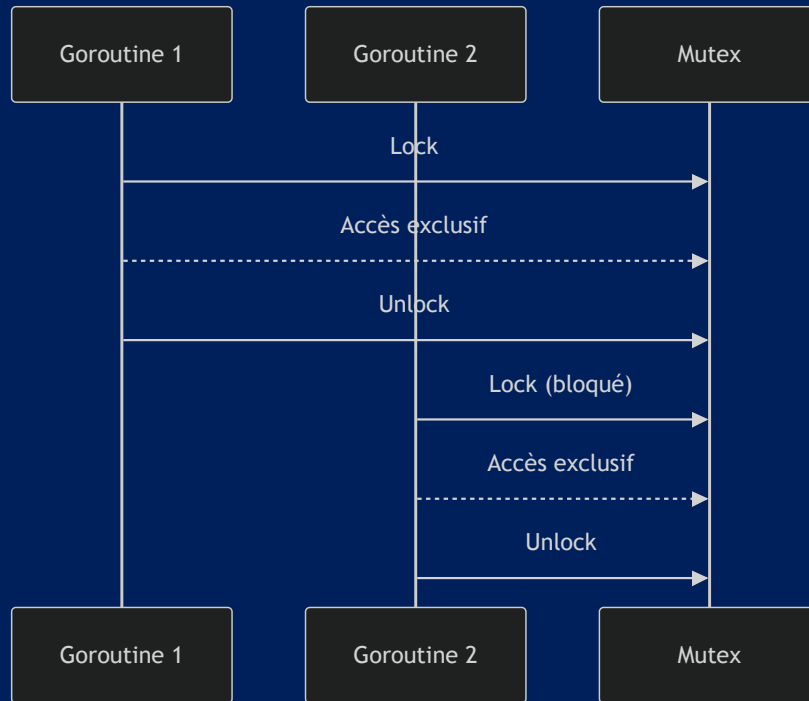
func main() {
    var wg sync.WaitGroup

    wg.Add(2)
    go deposit(100, &wg)
    go deposit(200, &wg)

    wg.Wait()
}
```

Comment un Mutex synchronise-t-il ?

Un diagramme de séquence pour visualiser l'interaction :



Points clés et ressources

Mécanisme	Description
<code>sync.Mutex</code>	Verrou d'exclusion mutuelle
<code>sync.WaitGroup</code>	Attente de fin de plusieurs goroutines
Protection	Permet d'éviter les conditions de course et les accès concurrents non contrôlés
Bonne pratique	Minimiser la section critique protégée

Sources pour aller plus loin :

- Documentation `sync` : <https://pkg.go.dev/sync>
- Go by Example - Mutex : <https://gobyexample.com/mutex>
- Go by Example - WaitGroup : <https://gobyexample.com/waitgroups>
- Go Blog - Sharing Memory by Communicating : <https://go.dev/blog/share>

Architecture Microservices : Principes Fondamentaux

Définition de l'architecture microservices

L'architecture microservices est un style d'architecture logicielle où une application est décomposée en **petits services autonomes** — chacun étant responsable d'une fonctionnalité métier précise.

Chaque microservice :

- Est développé, déployé et scalé **indépendamment**.
- Communique avec les autres via des API (souvent REST, gRPC...).
- Possède sa propre base de données ou modèle de stockage, assurant un découplage fort.

Avantages

Avantage	Description
Scalabilité	Échelle précise sur services spécifiques
Résilience	Défauts isolés à un service sans casser tout
Déploiement indépendant	Mises à jour plus rapides
Technologie mixte	Possibilité de différents stacks tech par service
Organisation agile	Équipes autonomes travaillant sur composantes distinctes

Inconvénients et défis

Problème	Description
Complexité accrue	Orchestration et gestion de nombreux services
Communication réseau	Latence, gestion des erreurs réseau
Transactions réparties	Gestion des données distribuées et consistance
Monitoring et debugging	Nécessité d'outils spécifiques

Pourquoi Go pour les microservices ?

Go est particulièrement adapté aux microservices pour plusieurs raisons :

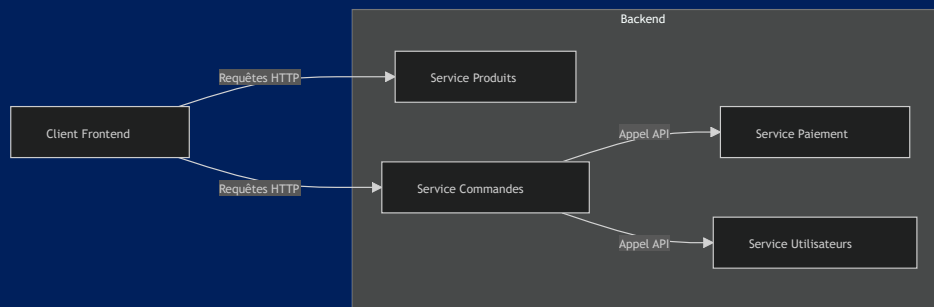
- **Performance élevée** avec compilation native.
- **Concurrence intégrée** avec goroutines idéale pour I/O asynchrone.
- **Binary statique** et taille réduite facilitant le déploiement en conteneur.
- **Écosystème riche** pour écrire des serveurs HTTP, clients gRPC, middlewares, etc.
- **Simplicité du langage** encourageant un code clair et maintenable.

Exemple simplifié d'architecture microservices

Imaginons une application e-commerce découpée en microservices :

- **Service Produits** (catalogue)
- **Service Commandes**
- **Service Paiement**
- **Service Utilisateurs**

Chaque service expose une API HTTP/REST ou gRPC.



Architecture typique microservices avec Go

- Chaque microservice écrit en Go possède :
 - Un serveur HTTP ou gRPC léger.
 - Des handlers/clients optimisés pour la communication inter-services.
 - Usage courant de **Docker** pour containerisation.
- Exemple d'une API Go simple exposée par un microservice (extrait) :

```
package main

import (
    "net/http"
    "github.com/gin-gonic/gin"
)

func main() {
    r := gin.Default()
    r.GET("/products/:id", func(c *gin.Context) {
        id := c.Param("id")
        // Simuler récupération produit
        c.JSON(http.StatusOK, gin.H{"id": id, "name": "Produit X"})
    })
    r.Run(":8080")
}
```

Communications entre microservices

- **REST/HTTP** : simple, largement supporté.
- **gRPC** : hautes performances, bonnes pratiques sur schémas protobuf.
- **Event-driven architectures** avec brokers (Kafka, NATS).

Go dispose de bibliothèques solides pour ces protocoles et middleware.

Monitoring et maintenance

L'architecture microservices nécessite :

- **Tracing distribué** (ex: Jaeger, OpenTelemetry)
- **Logs centralisés** (ex: ELK stack)
- **Health checks** intégrés dans les services.

Go facilite la production de ces informations via sa simplicité et richesse des outils.

Sources

- [Martin Fowler - Microservices](#)
- [Go for Microservices - Manning](#)
- [Gin Web Framework](#)
- [Google Cloud - Microservices avec Go](#)
- [gRPC Go](#)

Go et Microservices : Les Fondamentaux

Go est un langage de choix pour les microservices grâce à des atouts clés :

- **Performance & efficience :**
 - Binaire natif, rapide et léger.
 - Idéal pour les environnements contraints (containers, edge).
- **Concurrence native :**
 - Goroutines et channels facilitent la gestion d'I/O non bloquant et la parallélisation.
- **Simplicité & lisibilité :**
 - Code facile à maintenir, accélérant les cycles de développement.
- **Déploiement simple :**
 - Une exécutable unique simplifie le déploiement sans dépendances externes.
- **Écosystème riche :**
 - Nombreux frameworks et bibliothèques dédiés aux API, RPC, messaging...

Scénarios d'Application et Exemple Concret

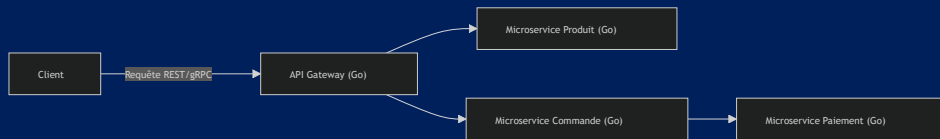
Cas d'usage typiques :

- Microservices exposant des API REST, gRPC ou event-driven.
- Traitement en arrière-plan et pipelines concurrents.
- Services scalables et rapidement déployables via containers (ex: Docker).

Conception d'une Architecture Microservices

Go facilite la construction d'architectures distribuées robustes et performantes.

Exemple d'architecture microservices typique avec Go :



Principes clés :

- Tous les services communiquent via HTTP/gRPC.
- Une API Gateway en Go gère le routage, l'authentification et l'agrégation.

Communication et Gestion Opérationnelle

Communication inter-services :

- **gRPC** : Solution performante et contractuelle, idéale pour la communication interne.
- **HTTP/REST** : Frameworks comme Gin, Echo, Fiber pour les API web.
- **Messaging & Event-driven** : Clients Go dédiés pour Kafka, NATS, RabbitMQ.

Gestion de la configuration et des dépendances :

- **Configuration** : Simplifiée avec des packages comme `viper` (fichiers, variables d'environnement).
- **Dépendances** : Gérées efficacement grâce au module Go (`go mod`).

Ces outils sont cruciaux pour des déploiements de microservices à grande échelle.

Visibilité et Robustesse en Production

Observabilité essentielle pour la production :

Go s'intègre bien avec les solutions modernes pour le suivi :

- **Tracing distribué** : Via OpenTelemetry.
- **Logging structuré** : Avec des outils comme logrus, zap.
- **Monitoring** : Intégration avec Prometheus, Grafana.

L'Essentiel à Retenir et Pour Aller Plus Loin

Ce qu'il faut retenir :

Go facilite la création et l'orchestration de microservices grâce à :

- Ses performances et sa légèreté.
- Ses primitives de concurrence natives.
- Son écosystème riche et sa simplicité syntaxique.
- Des exemples concrets comme les API REST et gRPC illustrent sa pertinence.

Ressources pour approfondir :

- Documentation officielle Go : <https://go.dev/doc/>
- Framework Gin : <https://gin-gonic.com/>
- gRPC Go documentation: <https://grpc.io/docs/languages/go/>
- Article DigitalOcean sur Go et microservices : <https://www.digitalocean.com/community/tutorials/how-to-build-a-microservice-architecture-with-golang>
- Blog Google Cloud sur microservices en Go : <https://cloud.google.com/blog/products/api-management/building-microservices-with-go>